The Bounded Cardinality Normal Form for the Logical Design of Relational Database Schemata

ZIHENG WEI, School of Computer Science, Wuhan University, Wuhan, China
SEBASTIAN LINK, School of Computer Science, The University of Auckland, Auckland, New Zealand

The goal of classical normalization is to maintain data consistency under updates, with a minimum level of effort. Given functional dependencies (FDs) alone, this goal is only achievable in the special case an FD-preserving Boyce–Codd Normal Form (BCNF) decomposition exists. As we show, in all other cases the level of effort can be neither controlled nor quantified. In response, we establish the ℓ-Bounded Cardinality Normal Form, parameterized by a positive integer ℓ. For every ℓ, the normal form condition requires from every instance that every value combination over the left-hand side of every non-trivial FD does not occur in more than ℓ tuples. BCNF is captured when $\ell = 1$. We show that schemata in ℓ-Bounded Cardinality Normal Form characterize instances in which updates to at most ℓ occurrences of any redundant data value are sufficient to maintain data consistency. In fact, for the smallest ℓ in which a schema is in ℓ-Bounded Cardinality Normal Form we capture an equilibrium between worst-case update inefficiency and best-case join efficiency, where some redundant data value can be joined with up to ℓ other data values. We then establish algorithms that compute schemata in ℓ-Bounded Cardinality Normal Form for the smallest level ℓ attainable across all lossless, FD-preserving decompositions. Additional algorithms (i) attain even smaller levels of effort based on the loss of some FDs, and (ii) decompose schemata based on prioritized FDs that cause high levels of effort. Our framework informs de-normalization already during logical design. In particular, every materialized view exhibits an equilibrium level ℓ that quantifies its worst-case incremental maintenance cost and its best-case support for join queries. Experiments with synthetic and real-world data illustrate which properties the schemata have that result from our algorithms, and how these properties predict the performance of update and query operations on instances over the schemata, without and with materialized views. We further demonstrate how our framework can automate the design of data warehouses by mining data for dimensions that exhibit high levels of data redundancy. In an effort to align data and the business rules that govern them, we use lattice theory to characterize ℓ-Bounded Cardinality Normal Form on database instances and schemata. As a consequence, any difference in constraints observed at instance and schema levels provides an opportunity to improve data quality, insight derived from analytics, and database performance.

CCS Concepts: • **Information systems** → **Database design and models**; **Relational database model**; **Integrity checking;**

Additional Key Words and Phrases: Cardinality constraint, data redundancy, functional dependency, join, normal form, normalization, update, view

This work is supported by the Marsden Fund Council from Government funding, managed by the Royal Society Te Apārangi under Grant No.: MFP-UOA24209.
Authors' Contact Information: Ziheng Wei, School of Computer Science, Wuhan University, Wuhan, Hubei, China; e-mail: ziheng.wei@whu.edu.cn; Sebastian Link, School of Computer Science, The University of Auckland, Auckland, Auckland, New Zealand; e-mail: s.link@auckland.ac.nz.

This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 0362-5915/2025/07-ART15
https://doi.org/10.1145/3744897

ACM Reference Format:
Ziheng Wei and Sebastian Link. 2025. The Bounded Cardinality Normal Form for the Logical Design of Relational Database Schemata. *ACM Trans. Datab. Syst.* 50, 4, Article 15 (July 2025), 45 pages. https://doi.org/10.1145/3744897

1 Introduction

Schema design aims at finding a layout of the data that facilitates the efficient processing of common queries and updates. The problem is challenging as data redundancy typically causes update inefficiency but promotes join efficiency. So far, the challenge has been addressed by performing normalization during logical schema design to achieve update efficiency, followed by denormalization during physical design to boost query efficiency. In particular, de-normalization occurs once the database is operational and patterns of data access on normalized databases emerge.

The goal of classical normalization is to maintain data consistency under updates, with a minimum level of effort. Normalization aims at eliminating any occurrence of redundant data values in any future database instance. This is attempted by structurally transforming **functional dependencies** (**FDs**), that cause data redundancy, into keys, that prohibit data redundancy. The well-known **Boyce–Codd Normal Form** (**BCNF**) requires the **left-hand side** (**LHS**) of every non-trivial FD to be a key. Hence, data redundancy can never be caused by FDs transformed into keys. However, while every schema can be decomposed into BCNF, FDs may be lost during this process and still cause data redundancy [6]. A more liberal condition is given by the **Third Normal Form** (**3NF**) where the LHS of every non-trivial FD must be a key or every attribute on the right-hand side must be prime (that is, to be part of some minimal key). 3NF synthesis can transform every schema into 3NF without losing any FD, but some FDs still cause data redundancy as they were not transformed into keys. Hence, classical normalization can only measure its success when no effort is required at all to maintain data consistency. This is only possible when an FD-preserving BCNF decomposition exists. In all other cases, the level of effort can be neither controlled nor even measured.

As running example consider the event management schema Hap with each record representing an *E(vent)* at a *V(enue)* and *T(ime)* with some *C(ompany)* in charge. Hap uses FDs to model business rules. The FD $E \rightarrow C$ says that only one company is in charge of every event, $V \rightarrow C$ says there cannot be different companies in charge at the same venue, $VT \rightarrow E$ says that no different events happen at the same venue at the same time, $ET \rightarrow V$ says that no event takes place at different venues at the same time, and $CT \rightarrow V$ expresses that no company is in charge of different venues at the same time. The minimal keys are ET, VT, and CT. Hence, every attribute of Hap is prime. While Hap is in 3NF it has no FD-preserving BCNF decomposition. Given FD-preservation, classical normalization cannot achieve more.

However, we cannot measure the level of effort to maintain data consistency for Hap. In instance r of Table 1a each of the $\ell_1 > 1$ occurrences of C-value *Kilo* is redundant due to the FD $E \rightarrow C$, and each of the $\ell_2 > 1$ occurrences of C-value *Mega* is redundant due to $V \rightarrow C$. Recall that a value occurrence is redundant whenever every update of the occurrence to a different value results in a violation of the FD. We refer to the number of different tuples in which a given data value can occur redundantly as the *level of update inefficiency* sufficient to maintain data consistency. Clearly, there is no finite level of update inefficiency that is sufficient to maintain data consistency on Hap. This is true for every schema that is in 3NF but not in BCNF. As we will show, every FD lost during BCNF decomposition still causes an unbounded level of update inefficiency. Indeed, changing one occurrence of *Kilo* means that a total of ℓ_1 occurrences of *Kilo* need updating to

Table 1. Schema Hap in 3NF and Its Decomposition $\mathcal{D}_1$ with Instances (Redundant Data Values in Bold)

(a) Instance r over Hap

Hap			
Event	*Venue*	*Company*	*Time*
Party	v_1	**Kilo**	t_1
⋮	⋮	⋮	⋮
Party	v_{ℓ_1}	**Kilo**	t_{ℓ_1}
e_1	Dome	**Mega**	t'_1
⋮	⋮	⋮	⋮
e_{ℓ_2}	Dome	**Mega**	t'_{ℓ_2}

(b) Decomposition $\mathcal{D}_1 = \{R_2, R_3, R_4\}$ of Hap with projection $\{r_i = \pi_{R_i}(r)\}_{i=2}^4$

R_2	
Venue	*Company*
v_1	Kilo
⋮	⋮
v_{ℓ_1}	Kilo
Dome	Mega

R_3		
Event	*Venue*	*Time*
Party	v_1	t_1
⋮	⋮	⋮
Party	v_{ℓ_1}	t_{ℓ_1}
e_1	Dome	t'_1
⋮	⋮	⋮
e_{ℓ_2}	Dome	t'_{ℓ_2}

R_4		
Event	*Company*	*Time*
Party	**Kilo**	t_1
⋮	⋮	⋮
Party	**Kilo**	t_{ℓ_1}
e_1	Mega	t'_1
⋮	⋮	⋮
e_{ℓ_2}	Mega	t'_{ℓ_2}

ensure data consistency. So, for FD-preserving BCNF decompositions the level of effort for data consistency is at optimum 1, while it is unbounded in all other cases.

In practice, however, ℓ_1 and ℓ_2 represent upper bounds of ***cardinality constraints*** **(CCs)**. They constitute a different class of business rules than FDs. For a positive integer ℓ, the CC $card(X) \leq \ell$ says that every instance can have up to ℓ different records with matching values on all the attributes in X. For $\ell = 1, X$ is a key. For $\ell = \infty$, no bound has been specified. For example, the CC $card(E) \leq \ell_1$ with $\ell_1 = 1,000$ ($1k$) expresses that every event can have up to $1k$ different combinations of venues and times. Similarly, the CC $card(V) \leq \ell_2$ with $\ell_2 = 1,000,000$ ($1m$) expresses that every venue can have up to $1m$ different combinations of events and times. CCs inform schema design beyond classical normalization. Given CCs, the 3NF schema Hap exhibits ℓ_2 as the smallest level of update inefficiency sufficient to maintain consistency. Without CCs no integer bounds are specified, and every non-trivial or lost FD causes an unbounded level of update inefficiency, unless its LHS is a key. Hence, we propose to include CCs in schema design.

With CCs we can quantify the level of effort required to maintain data consistency under updates. For example, we may ask which FD-preserving decompositions of Hap minimize the level ℓ of update inefficiency sufficient to maintain data consistency. Applying one of our new algorithms results in schema $\mathcal{D}_1 = \{(R_2, \Sigma_2), (R_3, \Sigma_3), (R_4, \Sigma_4)\}$ in Table 1b with

- $R_2 = CV$ and $\Sigma_2 = \{V \to C\}$,
- $R_3 = ETV$ and $\Sigma_3 = \{TV \to E, ET \to V\}$, and
- $R_4 = CET$ and $\Sigma_4 = \{CT \to E, E \to C\}$.

The schemata (R_2, Σ_2) and (R_3, Σ_3) are both in BCNF and cannot exhibit any redundant data value. However, R_4 is in 3NF and requires level $\ell_1 = 1k$ of update inefficiency to maintain data consistency, as caused by $E \to C$. Indeed, $\ell_1 = 1k$ is the smallest level achievable by any FD-preserving decomposition of Hap. The given FD $CT \to V$ is preserved by $\mathcal{D}_1$ as it is implied by $CT \to E$ and $ET \to V$. The decomposition $\mathcal{D}_2 = \{(R_1, \Sigma_1), (R_3, \Sigma_3), (R_5, \Sigma_5)\}$ with

- $R_1 = EC$ and $\Sigma_1 = \{E \to C\}$, and
- $R_5 = CTV$ and $\Sigma_5 = \{CT \to V, V \to C\}$

is in 3NF and requires level $\ell_2 = 1m$ of update inefficiency to maintain consistency.

CCs were already introduced in Chen's seminal ER paper [10] and used by UML, XML, and OWL. Surprisingly, they have not been used for normalization. We also observe that CCs quantify *the level of join efficiency* for a schema. We define the latter as the maximum number of values an FD can join with some redundant value in any instance of the schema. Instance r_4 over R_4 in Table 1b satisfies $E \to C$. Hence, r_4 is equal to the join $\pi_{EC}(r_4) \bowtie \pi_{ET}(r_4)$. Indeed, the redundant data C-value *Kilo* can be joined with the ℓ_1 T-values $t_1, \ldots, t_{\ell_1}$ via the E-value *Party*. Without CCs, the join efficiency is unbounded, so the classical theory does not provide insight into the join efficiency of

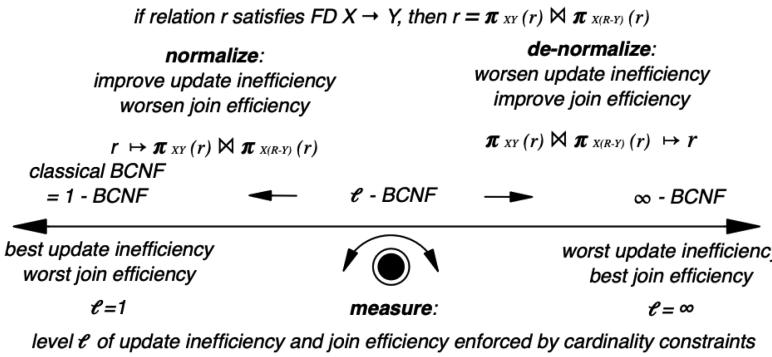


Fig. 1. Balancing worst-case update inefficiency and best-case join efficiency.

a schema. CCs inform the selection of materialized views, even before the database is operational. Here the smallest level ℓ of a materialized view quantifies both its worst-case effort required for incremental maintenance and its best-case support for joins. The smallest level of maintenance and join support of view Hap over $\mathcal{D}_1$ is $\ell_2 = 1m$.

Contributions. Our two main contributions are as follows:

— We establish the first framework for logical schema normalization that quantifies the level of data redundancy and update inefficiency of schemata.
— Our framework also quantifies the join efficiency of schemata, which means it informs de-normalization already at logical design time.

As the relational model forms the foundation for more expressive data formats, we expect our contributions to impact at least those data models for which schema design has been studied. These include SQL [32], nested relations [50], data warehouses [38], object-orientation [29], semantics [10], temporality [28], the Web [2], uncertainty [44], and graphs [19, 60, 61]. As a modern example of a data model where schemata are optional, property graphs have been shown to exhibit huge levels of data redundancy and potential for data inconsistency [62]. Similar to classical design, our results may be extended to higher normal forms such as 4NF [17], Project-Join Normal Form [18], or Inclusion-Dependency Normal Form [42].

Figure 1 illustrates our main ideas, in particular that the smallest level ℓ for which a schema is in ℓ-BCNF quantifies an equilibrium between the worst-case update effort and the best case join capability. Figure 2 illustrates our technical results, which are:

(1) We relax the classical BCNF condition by permitting every value combination over the LHS of every non-trivial FD to occur in up to ℓ tuples of every instance. Hence, we obtain the infinite hierarchy of ℓ-Bounded Cardinality Normal Forms (ℓ-BCNF), with classical BCNF captured for $\ell = 1$. Therefore, the minimum ℓ for which ℓ-BCNF is attainable measures the effort required to achieve data consistency.
(2) We show that schemata in ℓ-BCNF characterize instances for which level ℓ update inefficiency suffices to maintain data consistency. Schemata in ℓ-BCNF with minimum level ℓ exhibit instances that require level ℓ update inefficiency to maintain data consistency, but also permit level ℓ join efficiency.
(3) We establish an algorithm that computes schemata in ℓ-BCNF for the minimum level ℓ attainable across FD-preserving decompositions. Another algorithm prioritizes FDs for lossless decompositions based on the level of update inefficiency they cause. This algorithm produces

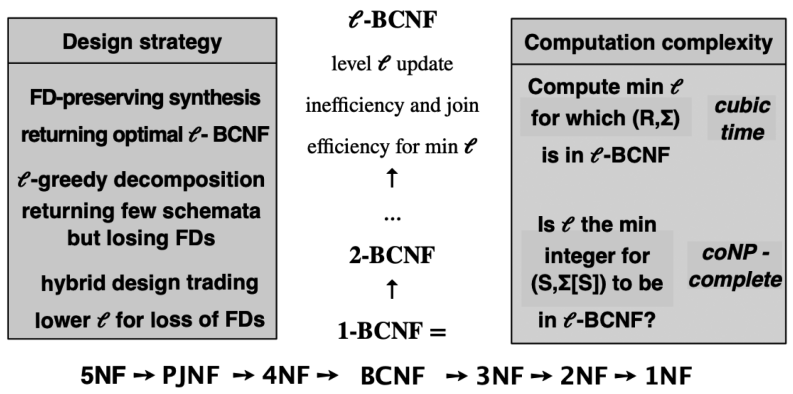


Fig. 2. Achievements of ℓ-BCNF.

few output schemata, but FDs may be lost. Combining both algorithms to a hybrid strategy, we further reduce the level attainable from FD-preserving decompositions by losing FDs.

(4) We inform the selection of materialized views as our normal forms can capture the worst-case incremental effort to maintain views under updates, and their best-case support for joins.
(5) Experiments illustrate which properties the schemata have that result from our algorithms, and how these properties predict the physical performance of update and query operations on instances over the schemata, without and with materialized views.
(6) Further experiments show how the use of CCs can inform the design of data warehouses, in particular dimension tables that eliminate large levels of data redundancy.
(7) Using lattice theory, we propose an algebraic framework for ℓ-BCNF that opens up the area for mathematical inquiry, similar to query optimization utilizing relational algebra, and enables practitioners to align their database instances with the business rules that govern them. Our characterizations of ℓ-BCNF are stated in terms of relation lattices for database instances and closure lattices for database schemata, respectively, and establish an isomorphism to Boolean algebras for every principal filter and ideal, respectively, whose level exceeds ℓ. The results provide a foundation for identifying (i) missing business rules or patterns of data, and (ii) invalid business rules or inconsistent data, which can prove useful in the evolution of database designs, higher quality data, and better performance.

Extension of conference version. The submission extends its conference version [45] in multiple ways. (1) The conference version did not contain any proofs. (2) A detailed background section makes our work accessible to non-specialists. (3) Extended examples and illustrations make it easier for readers to familiarize themselves with new concepts and exemplify to them what is being achieved by the general theory. (4) We have included a new experiment showcasing how the use of CCs promotes the quality of automated data warehouse designs. (5) The conference version did not contain any algebraic characterization, that is, Section 7 is entirely new material. (6) The related work section shows the relationship and differences to **numerical dependencies** (**NDs**) [22], the information-theoretical framework [35], and the algebraic characterization of BCNF for database instances [39].

Organization. We recall preliminaries in Section 2. In Section 3, we introduce our family of ℓ-BCNF and its computational properties. In Section 4, we establish which properties schemata in ℓ-BCNF exhibit in terms of updates and joins. We discuss three design algorithms in Section 5.

Table 2. Concepts and Their Notation in This Article

Concepts	
attribute A	domain of attribute dom(A)
relation schema $R = \{A_1, \ldots, A_n\}$	database schema $\mathcal{D} = \{R_1, \ldots, R_n\}$
tuple t	relation r
projection $t(X)$ of t on attribute set X	projection $r(X)$ of r on attribute set X
r satisfies FD $X \rightarrow Y$ ($\models_r X \rightarrow Y$)	r satisfies constraint set Σ ($\models_r \Sigma$)
Σ implies φ ($\Sigma \models \varphi$)	rule system $\mathfrak{R}$
inference of φ from Σ ($\Sigma \vdash_{\mathfrak{R}} \varphi$)	attribute set closure X^+_Σ of X under Σ
semantic closure Σ^* of constraint set Σ	syntactic closure $\Sigma^+_{\mathfrak{R}}$ of constraint set Σ
cover of constraint set Σ	non-redundant cover
L-reduced cover	canonical cover
(minimal) key X for (R, Σ)	prime attribute A of (R, Σ)
(lossless) decomposition	projection $\Sigma[S]$ of Σ to attribute set S
Boyce-Codd Normal Form (BCNF)	Third Normal Form (3NF)
BCNF (3NF) decomposition	FD-preserving decomposition

Experimental results are presented in Section 6. Algebraic foundations for ℓ-BCNF over instances and schemata are established in Section 7. Related work is discussed in Section 8. We conclude in Section 9. The online appendix, accessible in the ACM Digital Library, contains all proofs.

2 Preliminaries

In this section, we will present concepts and notation necessary to develop our framework. Most of these concepts are well-known [3, 4, 47], and a list of them is summarized in Table 2.

2.1 The Relational Model

A *relation schema* is a finite set R of attributes A, each having a domain dom(A) of possible values. A *database schema* is a finite set $\mathcal{D} = \{R_1, \ldots, R_n\}$ of relation schemata. A *tuple* t over R is a function $t : R \rightarrow \bigcup_{A \in R} dom(A)$ with $t(A) \in dom(A)$, that is, a tuple assigns to every attribute a value from its domain. A *relation* over R is a finite set r of tuples over R. The *projection* of a tuple t onto attribute set X is denoted by $t(X)$, and the projection of a relation r onto X is given by $r(X) = \{t(X) \mid t \in r\}$.

An FD over relation schema R is an expression $X \rightarrow Y$ with attribute sets $X, Y \subseteq R$. A relation r over R *satisfies* the FD $X \rightarrow Y$ over R whenever every pair of tuples $t, t' \in r$ that has values matching on all attributes in X also has values matching on all attributes in Y, that is, $\forall t, t' \in r(t(X) = t'(X) \Rightarrow t(Y) = t'(Y))$. Given a set Σ of constraints over relation schema R, a relation r over R satisfies Σ, denoted by $\models_r \Sigma$, if and only if for all $\sigma \in \Sigma$, $\models_r \sigma$. FDs of the form $X \rightarrow R$ over relation schema R denote *key dependencies*, and the attribute set X is called a *key*. Instead of saying that r satisfies the key dependency $X \rightarrow R$ we may simply say that r satisfies the key X. For example, the relation over Hap = {*E(vent)*, *V(enue)*, *C(ompany)*, *T(ime)*} in Table 1a satisfies the FDs $E \rightarrow C$ and $V \rightarrow C$.

2.2 Design Foundations

The success of Boyce–Codd and 3NF is promoted by the ability to decide the implication problem of FDs efficiently. The *(finite) implication problem* for a class C of constraints is to decide for every

Table 3. Armstrong Axioms for FDs

$\frac{}{XY \to Y}$	$\frac{X \to Y}{X \to XY}$	$\frac{X \to Y \quad Y \to Z}{X \to Z}$
(reflexivity, $\mathcal{R}$)	(extension, $\mathcal{E}$)	(transitivity, $\mathcal{T}$)

relation schema R and for every set $\Sigma \cup \{\varphi\}$ of constraints for C over R, whether Σ *(finitely) implies* φ, denoted by $\Sigma \models_{(f)} \varphi$. Here, Σ *(finitely) implies* φ iff every (finite) relation over R that satisfies Σ also satisfies φ. For the classes of constraints we consider, implication and finite implication coincide. Hence, we will speak of *the* implication problem. For example, given the set Σ_0 of FD $E \to C$, $V \to C$, $VT \to E$, $ET \to V$, and $CT \to V$, Σ_0 does imply the FD $CT \to E$.

For a class C of constraints, the *semantic closure* of a constraint set Σ over relation schema R is the set $\Sigma^* = \{\varphi \mid \Sigma \models \varphi\}$, which contains all constraints over R that are implied by Σ.

Two constraint sets Σ and Θ are *covers* of one another iff they imply the same constraints, that is, $\Sigma^* = \Theta^*$ [47]. An FD set Σ is said to be *non-redundant* iff for all $\sigma \in \Sigma$, $\Sigma - \{\sigma\} \not\models \sigma$, that is, none of the FDs in Σ is implied by the remaining FDs in the set. An FD set is *L-reduced* iff for all $X \to Y \in \Sigma$ there is no proper $Z \subset X$ such that $\Sigma \models Z \to Y$. Finally, a *canonical cover* for Σ is a cover of Σ that is non-redundant, L-reduced, and where the LHSs of all FDs are unique. For example, let Σ_0 denote the FDs $E \to C$, $V \to C$, $VT \to E$, $ET \to V$, $CT \to V$ and $CT \to E$, and Θ_0 denote the FDs in Σ_0 and $CT \to E$. Then Σ_0 is a canonical cover of Θ_0. While Θ_0 is L-reduced, the FD $CT \to E$ in Θ_0 is redundant.

Axiomatic and algorithmic solutions are well-known for the class of FDs [47]. In fact, we determine the semantic closure Σ^* of a set Σ by applying *inference rules* of the form $\frac{\text{premise}}{\text{conclusion}}$. For a set $\mathfrak{R}$ of inference rules let $\Sigma \vdash_{\mathfrak{R}} \varphi$ denote the *inference* of φ from Σ by $\mathfrak{R}$. That is, there is some sequence $\sigma_1, \ldots, \sigma_n$ such that $\sigma_n = \varphi$ and every σ_i is an element of Σ or is the conclusion that results from an application of an inference rule in $\mathfrak{R}$ to some premises in $\Sigma \cup \{\sigma_1, \ldots, \sigma_{i-1}\}$. Let $\Sigma^+_{\mathfrak{R}} = \{\varphi \mid \Sigma \vdash_{\mathfrak{R}} \varphi\}$ be the *syntactic closure* of Σ under inferences by $\mathfrak{R}$. $\mathfrak{R}$ is *sound* (respectively, *complete*) for class C iff for every set Σ of constraints from C over every relation schema R, we have $\Sigma^+_{\mathfrak{R}} \subseteq \Sigma^*$ (respectively, $\Sigma^* \subseteq \Sigma^+_{\mathfrak{R}}$). The (finite) set $\mathfrak{R}$ is a (finite) *axiomatization* if $\mathfrak{R}$ is both sound and complete. For example, the set $\mathfrak{A} = \{\mathcal{R}, \mathcal{E}, \mathcal{T}\}$ of inference rules in Table 3 denotes the Armstrong axioms, which form a finite axiomatization for the implication problem of FDs.

Axiomatizations can lead to efficient algorithms that decide implication. An FD $X \to Y$ is implied by an FD set Σ over relation schema R iff the *attribute set closure* $X^+_\Sigma = \{A \in R \mid \Sigma \models X \to A\}$ contains Y, that is, $Y \subseteq X^+_\Sigma$. This results in a linear-time algorithm for FD implication [3]. In our example, Σ_0 implies $CT \to E$ since $E \in \{C, T\}^+_{\Sigma_0} = \{C, E, T, V\}$.

2.3 Boyce–Codd Normal Form

For an FD set Σ over relation schema R, (R, Σ) is in BCNF iff for every FD $X \to Y \in \Sigma^+_{\mathfrak{A}}$ where $Y \not\subseteq X$, $X \to R \in \Sigma^+_{\mathfrak{A}}$ [47]. Since FDs $X \to Y$ where $Y \subseteq X$ are trivially satisfied by every relation, non-trivial FDs cause data redundancy unless they are key dependencies. Our schema $(\textsc{Hap}, \Sigma_0)$ is not in BCNF since the non-trivial FD $E \to C \in \Sigma$ is not a key dependency.

The property of being in BCNF is invariant under different choices of covers for Σ. For example, $(\textsc{Hap}, \Sigma_0)$ not being in BCNF is equivalent to $(\textsc{Hap}, \Theta_0)$ not being in BCNF since Σ_0 and Θ_0 are covers of one another. The invariance is due to using the syntactic closure in the BCNF condition. This, however, leaves open the question whether the BCNF condition can be tested efficiently. Indeed, (R, Σ) is in BCNF iff for every FD $X \to Y \in \Sigma$ where $Y \not\subseteq X$, $X \to R \in \Sigma^+_{\mathfrak{A}}$ [47]. Hence,

deciding whether (R, Σ) is in BCNF can be done in time quadratic in the input. However, for a sub-schema $S \subseteq R$ it is *coNP*-complete to decide whether $(S, \Sigma[S])$ is in BCNF, where $\Sigma[S] = \{X \to Y \in \Sigma^+_{\mathfrak{A}} \mid X, Y \subseteq S\}$ [3, 47]. For example, the schema $(R_5 = CTV, \Sigma_0[R_5])$ from the introduction is not in BCNF since the non-trivial FD $V \to C \in \Sigma_0[R_5]$ is not a key dependency over R_5.

A *decomposition* of relation schema R is a set $\mathcal{D}$ of relation schemata such that $\bigcup_{S \in \mathcal{D}} S = R$. A decomposition $\mathcal{D}$ of R with FD set Σ is *lossless* if for every relation r over R that satisfies Σ, $r = \bowtie_{S \in \mathcal{D}} r[S]$. Here, $r[S] = \{t(S) \mid t \in r\}$. A *BCNF decomposition* of R with FD set Σ is a decomposition $\mathcal{D}$ of R where for every $S \in \mathcal{D}$, $(S, \Sigma[S])$ is in BCNF. A decomposition $\mathcal{D}$ of (R, Σ) is *FD-preserving* if and only if $\forall \sigma \in \Sigma (\bigcup_{S \in \mathcal{D}} \Sigma[S] \models \sigma)$. For example, $\mathcal{D}_2$ from the introduction denotes a lossless decomposition of (Hap, Σ_0). While $(R_1, \Sigma_0[R_1] = \{E \to C\})$ and $(R_3, \Sigma_0[R_3] = \{VT \to E, ET \to V\})$ are in BCNF, $(R_5, \Sigma_0[R_5] = \{CT \to V, V \to C\})$ is not. This means, $\mathcal{D}_2$ is not a BCNF decomposition. We may split $(R_5, \Sigma_0[R_5])$ into $(R_2 = VC, \Sigma_0[R_2] = \{V \to C\})$ and $(R_6 = VT, \Sigma_0[R_6] = \emptyset)$, which are both in BCNF. Hence, $\mathcal{D}_0 = \{(R_1, \Sigma_0[R_1]), (R_2, \Sigma_0[R_2]), (R_3, \Sigma_0[R_3]), (R_6, \Sigma_0[R_6])\}$ would constitute a lossless BCNF decomposition of (Hap, Σ_0). Note that $\mathcal{D}_2$ is FD-preserving: The union of $\Sigma_0[R_2]$, $\Sigma_0[R_3]$, $\Sigma_0[R_5]$ implies all FDs in Σ_0. However, $\mathcal{D}_0$ is not FD-preserving since FD $CT \to V$ was lost.

Vincent introduced redundant data values and showed that BCNF is equivalent to not permitting any redundant data value in any instance that satisfies the given FDs [66]. In fact, a single occurrence of a data value is *redundant* whenever every change to this occurrence results in a relation that violates some given constraint. More formally, given relation r over relation schema R that satisfies a constraint set Σ, and given a tuple $t \in r$ and attribute $A \in R$, the value occurrence $t(A)$ is *redundant* iff every relation r' that results from r by replacing $t(A)$ by any other value will violate Σ. For example, replacing any single occurrence of Kilo in relation r of Table 1a by a different value will result in a relation that violates the FD $E \to C$.

2.4 Third Normal Form

Not every schema (R, Σ) permits a lossless, FD-preserving BCNF decomposition [47]. For example, (Hap, Σ_0) does not have a lossless, FD-preserving BCNF decomposition. For instance, $\mathcal{D}_0$ is a lossless BCNF decomposition that is not FD-preserving, and $\mathcal{D}_2$ is a lossless FD-preserving decomposition that is not in BCNF. In response, there is the 3NF proposal [7].

For an FD set Σ over relation schema R, (R, Σ) is in 3NF iff for every FD $X \to Y \in \Sigma^+_{\mathfrak{A}}$ where $Y \not\subseteq X$ holds, $X \to R \in \Sigma^+_{\mathfrak{A}}$ or every attribute in $Y - X$ is a prime attribute. Hence, every schema in BCNF is also in 3NF. An attribute $A \in R$ is *prime* iff it is contained in some minimal key, that is, $A \in K$ for some $K \to R \in \Sigma^+_{\mathfrak{A}}$ such that for all proper subsets $K' \subset K$, $K' \to R \notin \Sigma^+_{\mathfrak{A}}$. For example, $(R_5, \Sigma_0[R_5] = \{CT \to V, V \to C\})$ is in 3NF but not in BCNF. Indeed, the minimal keys on $(R_5, \Sigma_0[R_5])$ are CT and VT, so every attribute is prime.

A *3NF synthesis* of R with FD set Σ is a decomposition $\mathcal{D}$ of R where for every $S \in \mathcal{D}$, $(S, \Sigma[S])$ is in 3NF. The main idea behind 3NF was to minimize data redundancy while preserving all FDs [35]. As illustrated by our simple example, 3NF cannot guarantee any upper bound on the level of data redundancy. Validating whether a given schema (R, Σ) is in 3NF is *coNP*-complete due to the requirement of having to compute all minimal keys [3]. For example, the decompositions $\mathcal{D}_1$ and $\mathcal{D}_2$ are both lossless, FD-preserving 3NF decompositions of (Hap, Σ_0).

2.5 Normalization

A relation is the lossless join over its projections on XY and $X(R-Y)$ whenever the relation satisfies the FD $X \to Y$ [47, 57]. For example, since the relation r from Table 1a satisfies the FD $E \to C$, r is the lossless join over its projections on EC and EVT. That is, $r = \pi_{EC}(r) \bowtie \pi_{EVT}(r)$. This is illustrated in the following table.

r

Event	*Venue*	*Company*	*Time*
Party	v_1	Kilo	t_1
⋮	⋮	⋮	⋮
Party	v_{ℓ_1}	Kilo	t_{ℓ_1}
e_1	Dome	Mega	t'_1
⋮	⋮	⋮	⋮
e_{ℓ_2}	Dome	Mega	t'_{ℓ_2}

$=$

$\pi_{EC}(r)$

Event	*Company*
Party	Kilo
e_1	Mega
⋮	⋮
e_{ℓ_2}	Mega

$\bowtie$

$\pi_{EVT}(r)$

Event	*Venue*	*Time*
Party	v_1	t_1
⋮	⋮	⋮
Party	v_{ℓ_1}	t_{ℓ_1}
e_1	Dome	t'_1
⋮	⋮	⋮
e_{ℓ_2}	Dome	t'_{ℓ_2}

Hence, splitting (R, Σ) vertically into $(XY, \Sigma[XY])$ and $(X(R-Y), \Sigma[X(R-Y)])$ eliminates all redundant data values caused by $X \to Y$ over R, since X is a key on $(XY, \Sigma[XY])$. For example, all redundant occurrences of *Kilo* in r are eliminated in $\pi_{EC}(r)$ since E is a key over EC.

A BCNF decomposition starts with (R, Σ) and keeps splitting any schema $(S, \Sigma[S])$ that is not yet in BCNF. For example, $\{(EC, \{E \to C\}), (ETV, \{ET \to V, VT \to E\})\}$ is a lossless BCNF decomposition of (HAP, Σ_0) where the FDs $V \to C$ and $CT \to V$ have been lost.

Alternatively, whenever (R, Σ) is not in 3NF, *3NF synthesis* computes a canonical cover, and selects the output schemata as $(XY, \Sigma[XY])$ where XY is maximal under set containment among the FDs $X \to Y$ of the canonical cover, and adds a minimal key $(S, \Sigma[S])$, if still necessary, to remain lossless [7]. For example, Σ_0 is a canonical cover of itself, and $\mathcal{D}_2$ from the introduction constitutes a lossless, FD-preserving 3NF decomposition of (HAP, Σ_0) where $(R_5, \Sigma_0[R_5])$ is not in BCNF.

2.6 Design Foundations for Cardinality Constraints and Functional Dependencies

No previous schema design approach can quantify update inefficiency and join efficiency at the logical level. With FDs alone, BCNF decompositions cannot measure the level of data redundancy caused by lost FDs, and 3NF cannot measure the level of data redundancy caused by non-key FDs.

It is our observation here that CCs can be used to measure the level of data redundancy on database schemata at the logical level. This enables them to quantify the inherent tradeoff between update and join efficiency. CCs express important domain knowledge by stipulating limits on the number of tuples in which the same combination of values can occur [10, 24, 43, 64]. It is surprising that CCs have not been exploited for classical schema design. The reason why they have been looked at extensively in data modeling is because they occur frequently in practice [40, 58]. For example, it is natural to have upper bounds on resources such as the number of products that can be produced within a given time frame, the number of patients that can be cared for in parallel by staff, or the number of passengers that can be transported by vehicles over a period of time. To date the connection to schema design has simply not been made.

Let $\mathbb{N}^{\infty}_{\geq 1}$ denote the union of the positive integers $\mathbb{N}_{\geq 1}$ and ∞. A CC over relation schema R is an expression $card(X) \leq \ell$ where $X \subseteq R$ and $\ell \in \mathbb{N}^{\infty}_{\geq 1}$. A relation r over R satisfies $card(X) \leq \ell$ whenever there are no more than ℓ different tuples in r that all have matching values on all the attributes in X, that is, $\forall t_1, \ldots, t_{\ell+1} \in r(t_1(X) = \ldots = t_{\ell+1}(X) \Rightarrow \exists i, j \in \{1, \ldots, \ell+1\}(i \neq j \wedge t_i = t_j))$. For example, the relation in Table 1a satisfies $card(EC) \leq \ell_1 = 1k$ but violates $card(EC) \leq 999$ due to the $1k$ different tuples that have E-value *Party* and C-value *Kilo*.

Importantly, CCs subsume the class of keys as the special case where $\ell = 1$. Indeed, a relation satisfies the key X if and only if the relation satisfies the CC $card(X) \leq 1$.

Since we want to extend the current schema design framework from FDs alone to the combined class of CCs and FDs, we need to address the implication problem for this combined class. As before, finite and unrestricted implication problems for the combined class do coincide.

Table 4 shows different axiomatizations. The system $\mathfrak{H} = \{\mathcal{S}, \mathcal{U}, \mathcal{L}, \mathcal{A}\}$ forms an axiomatization for CCs alone [24], and the system $\mathfrak{L} = \mathfrak{H} \cup \mathfrak{A} \cup \{\mathcal{K}, \mathcal{P}\}$ forms an axiomatization for CCs and FDs

Table 4. Axiomatizations of CCs and FDs: $\mathfrak{H} = \{\mathcal{S}, \mathcal{U}, \mathcal{L}, \mathcal{A}\}$ Forms an Axiomatization for CCs Alone, $\mathfrak{A}$ from Table 3 Forms an Axiomatization for FDs Alone, and $\mathfrak{Q} = \mathfrak{H} \cup \mathfrak{A} \cup \{\mathcal{K}, \mathcal{P}\}$ Forms an Axiomatization for the Combined Class of CCs and FDs

$\frac{}{card(R) \leq 1}$ (set, $\mathcal{S}$)	$\frac{}{card(X) \leq \infty}$ (unbounded, $\mathcal{U}$)	$\frac{card(X) \leq \ell}{card(X) \leq \ell + 1}$ (loosen, $\mathcal{L}$)	$\frac{card(X) \leq \ell}{card(XY) \leq \ell}$ (adding, $\mathcal{A}$)
$\frac{card(X) \leq 1}{X \to Y}$ (key, $\mathcal{K}$)	$\frac{X \to Y \quad card(Y) \leq \ell}{card(X) \leq \ell}$ (pullback, $\mathcal{P}$)		

together [24], where $\mathfrak{A}$ denotes the Armstrong axioms from Table 3. For example, given the CC $card(EC) \leq 1,000$ and the FD $E \to C$ we can use the following inference of $card(E) \leq 1,000$ using the axiomatization $\mathfrak{Q}$.

$$\dfrac{\dfrac{E \to C}{\mathcal{E}: \quad E \to EC} \qquad card(EC) \leq 1,000}{\mathcal{P}: \qquad card(E) \leq 1,000}$$

Though more expressive than CCs and FDs in isolation, the combined implication problem can be reduced to that for FDs alone by translating any set Σ of CCs and FDs into the FD set [24]:

$$\Sigma[FD] = \{X \to Y \mid X \to Y \in \Sigma\} \cup \{X \to R \mid \text{card}(X) \leq 1 \in \Sigma\}.$$

The following result from [24] is an effective reduction to the implication problem of FDs alone.

Theorem 2.1 ([24]). *For a given set Σ of CCs and FDs over a relation schema R, the following hold:*

(1) $\Sigma \models X \to Y$ if and only if $\Sigma[FD] \models X \to Y$

(2) For every positive integer ℓ, $\Sigma \models card(X) \leq \ell$ if and only if $Y \subseteq X^+_{\Sigma[FD]}$ for some $card(Y) \leq \ell' \in \Sigma \cup \{card(R) \leq 1\}$ where $\ell' \leq \ell$.

For example, if Σ contains $E \to C$ and $card(EC) \leq 1k$, then $card(E) \leq 2k$ is implied by Σ since $E^+_{\Sigma[\text{FD}]} = EC$ and, therefore, $EC \subseteq E^+_{\Sigma[\text{FD}]}$ for the CC $card(EC) \leq 1k \in \Sigma$ and $\ell' = 1k \leq 2k = \ell$.

So, deciding implication for CCs and FDs combined is in time $O(|\Sigma| \times ||\Sigma||)$ where $|\Sigma|$ is the number of elements in Σ and $||\Sigma||$ the total number of attribute occurrences in Σ [24].

In the following we will lift the existing logical normalization framework from the individual class of FDs to the combined class of CCs and FDs. The goal is to quantify the level of effort required to maintain consistency under update operations.

3 The Family of ℓ-BCNF

We establish the family of ℓ-Bounded Cardinality Normal Forms and some computational properties. The levels of data redundancy and update inefficiency prevented by schemata in this normal form and the level of join efficiency permitted by schemata in this normal form are studied in Section 4.

A schema is in BCNF iff the LHS X of every non-trivial FD $X \to Y$ is a key. Since X is a key iff $card(X) \leq 1$ holds, CCs lend themselves for relaxing the BCNF condition as follows.

Definition 3.1. Let Σ denote a set of CCs and FDs over relation schema R, and let $\ell \in \mathbb{N}^\infty_{\geq 1}$. (R, Σ) is in *ℓ-Bounded Cardinality Normal Form* (ℓ-BCNF) if and only if for all $X \to Y \in \Sigma^+_{\mathfrak{Q}}$ where $Y \not\subseteq X$ we have that $card(X) \leq \ell \in \Sigma^+_{\mathfrak{Q}}$.

For $\ell = 1$, a schema is in ℓ-BCNF iff it is in BCNF. In this case, no redundant data value can ever occur in any relation. Intuitively, with larger ℓ, schemata in ℓ-BCNF permit relations with higher levels of data redundancy.

Example 3.2. For our running example Hap consider the FD set $\overline{\Sigma}_0$ of $E \to C$, $V \to C$, $VT \to E$, $ET \to V$, and $CT \to V$. While $(\text{Hap}, \overline{\Sigma}_0)$ is in 3NF, it is in ∞-BCNF as the smallest ℓ for which $card(E) \leq \ell \in (\overline{\Sigma_0})^+_{\mathfrak{Q}}$ is $\ell = \infty$. Intuitively, this matches our understanding that the levels of data redundancy, update inefficiency, and join efficiency are unbounded. Consider now (Hap,Σ_0) where Σ_0 consists of the FDs in $\overline{\Sigma}_0$ and the CCs $card(E) \leq 1k$ and $card(V) \leq 1m$. Since VT, ET, and CT are (minimal) keys, we obtain the CCs $card(VT) \leq 1$, $card(ET) \leq 1$, and $card(CT) \leq 1$. Hence, every non-trivial $X \to Y \in \Sigma_0$ satisfies $card(X) \leq 1m \in (\Sigma_0)^+_{\mathfrak{Q}}$. The latter condition is equivalent to being in ℓ-BCNF (Theorem 3.5), so (Hap, Σ_0) is in $1m$-BCNF. (Hap, Σ_0) is not in any ℓ-BCNF for $\ell < 1m$ since we have the FD $V \to C \in \Sigma_0$ but the smallest ℓ_2 with $card(V) \leq \ell_2 \in (\Sigma_0)^+_{\mathfrak{Q}}$ is $\ell_2 = 1m$.

Our definition of ℓ-BCNF has the desirable property to be invariant under covers, as stated next.

Proposition 3.3. *Let Σ and Θ denote two CC/FD sets over R that are covers of one another. For all $\ell \in \mathbb{N}^{\infty}_{\geq 1}$, (R, Σ) is in ℓ-BCNF iff (R, Θ) is in ℓ-BCNF.*

Following Proposition 3.3 we need not worry how to represent application semantics by integrity constraints. If Σ'_0 denotes the FDs in $\overline{\Sigma}_0$ together with the CCs $card(EC) \leq 1k$ and $card(VC) \leq 1m$, then Σ_0 and Σ'_0 are covers of one another, as easily seen from the adding rule $\mathcal{A}$, the extension rule $\mathcal{E}$, and the pullback rule $\mathcal{P}$. Hence, (Hap,Σ'_0) is in $1m$-BCNF but not in ℓ-BCNF for any $\ell < 1m$.

Our definition gives rise to a family of syntactic normal forms that exhibit a strict hierarchy.

Theorem 3.4. *For every schema (R, Σ) and every positive integer ℓ we have: If (R, Σ) is in ℓ-BCNF, then (R, Σ) is also in $\ell + 1$-BCNF. However, for every positive integer ℓ there are schemata (R, Σ) in $\ell + 1$-BCNF that are not in ℓ-BCNF.*

For example, (Hap, Σ_0) is in ℓ-BCNF iff $\ell \geq 1m$. Theorem 3.4 establishes the first infinite hierarchy of normal forms for relational schema design. It is orthogonal to known normal forms such as 3NF, BCNF, 4NF, and so on. We now turn to the computational complexity of problems associated with schemata in ℓ-BCNF. It turns out that combining CCs and FDs raises expressiveness without adding (much) complexity. As a generalization of BCNF, known hardness results for BCNF apply to ℓ-BCNF.

3.1 Efficient Decidability Locally

Firstly, we show that deciding whether for a given schema (R, Σ) and for a given positive integer ℓ, (R, Σ) is in ℓ-BCNF can be done in cubic time in the input. This is a consequence of showing that it suffices to check the FDs in Σ to validate whether (R, Σ) is in ℓ-BCNF.

Theorem 3.5. *For all (R, Σ) and $\ell \in \mathbb{N}^{\infty}_{\geq 1}$, (R, Σ) is in ℓ-BCNF iff for all $X \to Y \in \Sigma$ where $Y \not\subseteq X$, we have that $card(X) \leq \ell \in \Sigma^+_{\mathfrak{Q}}$. We can decide in $O(|\Sigma|^2 \times ||\Sigma||)$ time whether for a given schema (R, Σ) and a given $\ell \in \mathbb{N}^{\infty}_{\geq 1}$, (R, Σ) is in ℓ-BCNF.*

3.2 Computing the Strongest Local Normal Form

We want to compute the smallest positive integer ℓ for which a given schema (R, Σ) is in ℓ-BCNF. We use Σ_{FD} to denote the set of non-trivial FDs in Σ, and Σ_{CC} to denote the set of non-trivial CCs in Σ together with the CC $card(R) \leq 1$, since R is always a key. Algorithm 1 computes the smallest ℓ for which (R, Σ) is in ℓ-BCNF. The steps are as follows.

ALGORITHM 1: Strongest ℓ-Bounded Cardinality Normal Form

Require: (R, Σ) with CC/FD set Σ over schema R
Ensure: Minimum integer ℓ such that (R, Σ) in ℓ-BCNF
1: $\Sigma_{\text{FD}} \leftarrow \{X \rightarrow Y \in \Sigma \mid Y \not\subseteq X\}$
2: $\Sigma_{\text{CC}} \leftarrow \{card(X) \leq \ell \in \Sigma \mid \ell \neq \infty\} \cup \{card(R) \leq 1\}$
3: **if** $\Sigma_{\text{FD}} = \emptyset$ **then**
4: **return** 1
5: **for all** $X \rightarrow Y \in \Sigma_{\text{FD}}$ **do**
6: **if** ℓ_X has not been defined before **then**
7: $\ell_X \leftarrow \infty$
8: **for all** $card(Y) \leq \ell \in \Sigma_{\text{CC}}$ with $\ell < \ell_X$ **do**
9: **if** $Y \subseteq X^+_{\Sigma[\text{FD}]}$ **then**
10: $\ell_X \leftarrow \ell$
11: **if** $\ell_X = \infty$ **then**
12: **return** ∞
13: **return** $\max\{\ell_X \mid \ell_X \text{ is defined}\}$

The first two lines define Σ_{FD} and Σ_{CC}. If there are only trivial FDs in the input (line 3), the output level will be 1 (line 4). For each non-trivial input FD with LHS X (line 5) we determine the smallest positive integer ℓ_X by which X is bound (line 5–12). Initially, ℓ_X is ∞ (line 6–7), and any current ℓ_X can be replaced by a smaller ℓ (line 10) if a CC $card(Y) \leq \ell \in \Sigma_{\text{CC}}$ is found (line 8) such that $X \rightarrow Y$ is implied (line 9). In that case, $card(X) \leq \ell$ is also implied by Σ (see rule $\mathcal{P}$ from Table 4). We terminate with ∞ if we find that $\ell_X = \infty$ for some X (line 11–12). Otherwise, we return the maximum ℓ_X among those computed (line 13).

Due to Theorem 2.1, Algorithm 1 computes the strongest ℓ-BCNF possible. Essentially, for each non-trivial FD $X \rightarrow Y$ in the input we check for every non-trivial CC in the input whether it implies some smaller bound ℓ_X. Algorithm 1 therefore operates in $O(|\Sigma|^2 \times ||\Sigma||)$ time. We thus obtain the following result.

Corollary 3.6. *Given a set Σ of CCs and FDs over relation schema R, we can compute in $O(|\Sigma|^2 \times ||\Sigma||)$ time the smallest positive integer ℓ such that (R, Σ) is in ℓ-BCNF.*

For (Hap, Σ_0), $\ell_E = 1k$, $\ell_V = 1m$, $\ell_{VT} = 1$, $\ell_{ET} = 1$, and $\ell_{CT} = 1$. Since the maximum is $\ell = \ell_V = 1m$, ℓ_V is the optimum ℓ for which (Hap, Σ_0) is in ℓ-BCNF.

3.3 Likely Intractability Globally

While efficient locally, it is unlikely efficient to compute a strongest normal form globally. Given (R, Σ, ℓ) and some $S \subset R$, there is likely no PTIME algorithm deciding if $(S, \Sigma[S])$ is in ℓ-BCNF.

Theorem 3.7. *Let Σ denote a set of CCs and FDs over R, $S \subset R$, and ℓ a positive integer. Given (R, S, Σ, ℓ), it is* coNP*-complete to decide whether $(S, \Sigma[S])$ is in ℓ-BCNF.*

While it is already *coNP*-complete to decide if a given subschema is in BCNF, the parameter ℓ encourages us to understand schema normalization as the quest of computing a design that requires the least effort possible to maintain data integrity.

4 Useful Properties of ℓ-BCNF

We show that schemata in ℓ-BCNF characterize instances with useful properties for updates and joins. In particular, ℓ quantifies the highest number of values that need updating to achieve data consistency, but also how many data values can be joined with a given redundant value.

4.1 ℓ-redundancy

Intuitively, Vincent [66] defined a single occurrence of a data value as *redundant* whenever every change to this occurrence results in a relation that violates some given constraint. Our idea is to fix some positive integer ℓ, and define a data value as *ℓ-redundant* whenever there are ℓ distinct tuples in which the value occurs and every update to at least one of these ℓ occurrences results in a relation that violates a given constraint. That is, the value of these ℓ occurrences is already uniquely determined by the other values and knowing the constraints are satisfied by the relation.

For example, the value *Kilo* in relation r_4 in Table 1a is 999- but not $1k$-redundant ($\ell_1 = 1k$): Concealing between 1 and 999 occurrences of the value *Kilo* in the C-column still allows us to determine each of the concealed occurrences as $E \rightarrow C$ must hold and there is at least one tuple left that has a matching E-value and C-value *Kilo*, as illustrated next.

E	C	T
Party	?	t_1
⋮	⋮	⋮
Party	?	t_{999}
Party	Kilo	t_{1k}

$E \rightarrow C$
? = Kilo

E	C	T
Party	Kilo	t_1
⋮	⋮	⋮
Party	Kilo	t_{999}
Party	Kilo	t_{1k}

However, concealing all $1k$ occurrences means those tuples could all share any value on C.

Let Σ denote a set of CCs and FDs over a relation schema R. Given a relation r over R with ℓ distinct tuples $t_1, \ldots, t_\ell \in r$ and some attribute $A \in R$, we define an *ℓ-transaction* of $t_1, \ldots, t_\ell$ for A as a set $\{t'_1, \ldots, t'_\ell\}$ of tuples over R such that for all $i = 1, \ldots, \ell$ and for all $A' \in R - \{A\}$, $t'_i(A') = t_i(A')$, and there is some $j \in \{1, \ldots, \ell\}$ such that $t'_j(A) \neq t_j(A)$. In other words, an ℓ-transaction causes an actual update to at least one of the ℓ values $t_1(A), \ldots, t_\ell(A)$ and at most all of them. We are now prepared to define the concept of an ℓ-redundant data value occurrence.

Definition 4.1. Let Σ denote a set of CCs and FDs over a relation schema R, r a relation over R that satisfies Σ, $A \in R$, and ℓ a positive integer. The data value $v \in r(A)$ is *ℓ-redundant* for Σ iff there are ℓ distinct tuples $t_1, \ldots, t_\ell \in r$ such that $t_1(A) = \ldots = t_\ell(A) = v$, and for every ℓ-transaction $\{t'_1, \ldots, t'_\ell\}$ of $t_1, \ldots, t_\ell$ for A, the relation $r' := (r - \{t_1, \ldots, t_l\}) \cup \{t'_1, \ldots, t'_\ell\}$ violates some $\sigma \in \Sigma$.

We define a family of semantic normal forms by stipulating the absence of any relations that feature any ℓ-redundancy, for every positive integer ℓ.

Definition 4.2. Let Σ denote a set of CCs and FDs over relation schema R, and let $\ell \in \mathbb{N}_{\geq 1}$. Then (R, Σ) is in ***ℓ-Redundancy Free Normal Form*** (***ℓ-RFNF***) iff there is no relation r over R that satisfies Σ and there is no attribute $A \in R$ for which there is a data value $v \in r(A)$ that is ℓ-redundant for Σ.

If (R, Σ) is in *ℓ-RFNF*, then ℓ is a *level of data redundancy that* (R, Σ) *prevents*. Otherwise, (R, Σ) *permits this level*. If there is no positive integer ℓ for which (R, Σ) is in *ℓ-RFNF*, then (R, Σ) permits a level of data redundancy that is a priori unbounded. In this case, $\ell := \infty$.

(Hap, Σ_0) from Example 3.2 is in ℓ-RFNF iff $\ell \geq \ell_2 = 1m$. Indeed, every occurrence of *Mega* in relation r from Table 1a is ℓ-redundant for every $\ell < \ell_2$. Concealing fewer than the ℓ_2 occurrences of *Mega* still allows us to deduce their value from the remaining values and the fact that Σ_0 needs to be satisfied. The level of data redundancy that (Hap, $\overline{\Sigma}_0$) permits is a priori unbounded, since $\overline{\Sigma}_0$ contains non-trivial non-key FDs but not any CCs.

4.2 ℓ-update Inefficiency

Preventing ℓ-redundant data value occurrences is synonymous with maintaining data consistency with level ℓ-update inefficiency. Indeed, preventing ℓ-redundant data value occurrences means

that there is some ℓ-transaction that can consistently update a current relation. Take, for instance, the table on the right above. Updating all 1,000 occurrences of *Kilo* consistently to a new value will satisfy $E \rightarrow C$. Based on the CC $card(E) \leq 1k$, we require the update of at most 1,000 tuples to maintain consistency for $E \rightarrow C$, in any relation that satisfies the given constraints. Hence, it is justified to say that (R, Σ) is in ℓ-***Update Inefficiency Normal Form*** (**UINF**) iff (R, Σ) is in ℓ-RFNF.

If (R, Σ) is in ℓ-*UINF*, then ℓ is a *level of update inefficiency that is sufficient for* (R, Σ) *to maintain data consistency*. Otherwise, the level of update inefficiency is insufficient for (R, Σ) *to maintain data consistency*. Furthermore, if (R, Σ) is in ℓ-*UINF* for the smallest possible ℓ, then we also say that ℓ is the level of update inefficiency that is required for (R, Σ) to maintain data consistency. If there is no positive integer ℓ for which (R, Σ) is in ℓ-*UINF*, the level of update inefficiency required for (R, Σ) to maintain data consistency is a priori unbounded. In this case, $\ell := \infty$.

4.3 ℓ-join Efficiency

We have just seen how ℓ-redundancy can be interpreted as an ℓ-update inefficiency. Indeed, whenever ℓ-redundancy cannot occur, ℓ updates suffice to maintain consistency, and vice versa.

We will see now that the minimum level of ℓ-redundancy preventable also quantifies the maximum level of join efficiency attainable by any relation that satisfies the given constraint set over a given schema. Intuitively, we define join efficiency as the maximum number of tuples that have matching values on the LHS of a non-trivial FD. As a consequence, we are able to quantify both the update and join efficiency due to FDs, which is a core tradeoff for logical schema design.

Formally, if r satisfies the non-trivial FD $X \rightarrow Y$ over relation schema R, then r is the lossless join of its projections on XY and $X(R - Y)$. That is, $r = \pi_{XY}(r) \bowtie \pi_{X(R-Y)}(r)$. Intuitively, the X-value of any tuple t in r joins the value $t(Y)$ with the $R-Y$-value of all the tuples $t_1, \ldots, t_\ell$ in r that have matching values with t on X. This number ℓ denotes the join strength of t, given $X \rightarrow Y$.

Definition 4.3. Let r denote a relation that satisfies the set Σ of CCs and FDs over relation schema R. The *join strength* of a tuple t in r with respect to a non-trivial FD $\sigma = X \rightarrow Y \in \Sigma^+_{\mathcal{Q}}$ is the positive integer $\ell^{\sigma}_{t,r}$ that denotes the number of tuples in r that have matching values with t on all the attributes in X, that is, $\ell^{\sigma}_{t,r} = |\{t' \in r \mid t'(X) = t(X)\}|$.

For example, let $\Sigma = \{CT \rightarrow E, E \rightarrow C, card(E) \leq 1k\}$ denote a set of CCs and FDs over relation schema $\{E, C, T\}$. The following relation r satisfies Σ, in particular $E \rightarrow C$ and is therefore its lossless join over its projections $r(EC)$ and $r(ET)$.

E	C	T
Party	Kilo	t_1
⋮	⋮	⋮
Party	Kilo	t_{1k}

$=$

E	C
Party	Kilo

$\bowtie$

E	T
Party	t_1
⋮	⋮
Party	t_{1k}

The join strength of every tuple in r with respect to $E \rightarrow C$ is $1k$ since the relation joins the redundant C-value *Kilo* with the $1k$ different values $t_1, \ldots, t_{1k}$.

Definition 4.4. For a set Σ of CCs and FDs over relation schema R, (R, Σ) is in ℓ-***Join Efficiency Normal Form*** (**JENF**) iff there is some relation r over R that satisfies Σ, there is some non-trivial FD $\sigma \in \Sigma^+_{\mathcal{Q}}$, and there is some tuple $t \in r$ such that the join strength of t in r with respect to σ is ℓ.

If (R, Σ) is in ℓ-*JENF*, then ℓ is a *level of join efficiency that* (R, Σ) *permits*. Otherwise, ℓ is a *level of join efficiency that* (R, Σ) *prevents*. If there is no positive integer ℓ for which (R, Σ) is in ℓ-*JENF*, then the level of join efficiency that (R, Σ) permits is a priori unbounded. In this case, $\ell := \infty$.

In our example, (ECT, Σ) is in ℓ-JENF if and only if $\ell \leq 1k$. Hence, (ECT, Σ) permits a level of join efficiency up to $1k$, and prevents any level of join efficiency above $1k$.

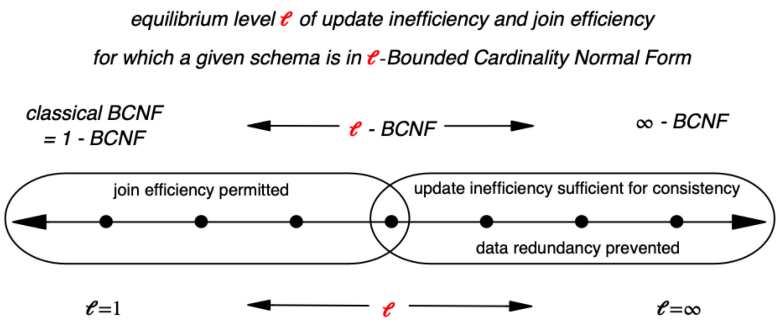


Fig. 3. Equilibrium level ℓ that captures both the smallest level of update inefficiency sufficient to maintain data consistency and the largest level of join efficiency achievable, which is syntactically characterized by the smallest level for which a given schema is in ℓ-Bounded Cardinality Normal Form

4.4 Justification

It turns out for every $\ell \in \mathbb{N}_{\geq 1}^{\infty}$ that ℓ-BCNF coincides with ℓ-RFNF and ℓ-UINF. In addition, the smallest ℓ for which a schema is in ℓ-BCNF coincides with the largest level for which the schema is in ℓ-JENF. Hence, schemata that are in ℓ-BCNF for the smallest ℓ possible exhibit instances that are all of the following: (i) free from any ℓ-redundant data value occurrences, (ii) ℓ-update inefficiency is required to maintain data consistency and (iii) level ℓ-join efficiency is permitted. First, we establish the equivalence between ℓ-BCNF and ℓ-RFNF (ℓ-UINF), for every ℓ.

Theorem 4.5. *For all (R, Σ) and all $\ell \in \mathbb{N}_{\geq 1}^{\infty}$, the following are equivalent:*
(1) (R, Σ) is in ℓ-RFNF (ℓ-UINF)
(2) (R, Σ) is in ℓ-BCNF.

We show next how our framework is capable of quantifying the tradeoff between update and join efficiency for every schema with a set of CCs and FDs. Indeed, the minimum level of update inefficiency that is sufficient for a schema to maintain data consistency coincides with the maximum level of join efficiency that the schema permits.

Theorem 4.6. *For every relation schema R and for every set Σ of CCs and FDs over R, the minimum level of update inefficiency that is sufficient for (R, Σ) to maintain data consistency is the maximum level of join efficiency that (R, Σ) permits.*

As a consequence of Theorems 4.5 and 4.6, every schema exhibits a unique level ℓ that quantifies the equilibrium between the smallest level of update inefficiency sufficient for data consistency maintenance and the largest join efficiency that is attainable.

Corollary 4.7. *For every relation schema R and for every set Σ of CCs and FDs over R there is a unique level $\ell \in \mathbb{N}_{\geq 1}^{\infty}$ such that (R, Σ) is in (i) ℓ-Bounded Cardinality Normal Form, (ii) ℓ-RFNF (UINF), and (iii) ℓ-JENF.*

We call the unique level ℓ in Corollary 4.7 *the equilibrium between update inefficiency and join efficiency exhibited by schema* (R, Σ). Hence, the equilibrium captures both the smallest level of update inefficiency sufficient to maintain data consistency and the largest level of join efficiency achievable, and this level is syntactically characterized by the smallest level for which the schema is in Bounded Cardinality Normal Form. This situation is illustrated in Figure 3.

As an illustration, the equilibrium level of $(R_4 = ECT, \Sigma_4 = \{CT \rightarrow E, E \rightarrow C, card(E) \leq 1000\})$ is $\ell_1 = 1,000$. Indeed, the following relation

E	C	T
Party	Kilo	t_1
⋮	⋮	⋮
Party	Kilo	$t_{1,000}$

is a relation over (R_4, Σ_4) that (i) is free of level 1,000 data redundancy, (ii) level 1,000 update inefficiency is sufficient to maintain consistency, and (iii) exhibits level 1,000 join efficiency. (R_4, Σ_4) is also in 1,001-BCNF, and every relation over (R_4, Σ_4) is free of 1,001 data redundancy and level 1,001 update inefficiency is sufficient to maintain data consistency, but there is no relation over (R_4, Σ_4) in which level 1,001 join efficiency can be exhibited.

In our running example, (Hap, Σ_0) from Example 3.2 is in ℓ_2-BCNF but not in $\ell_2 - 1$-BCNF. Indeed, the CC $card(V) \leq \ell_2 - 1$ is not implied by Σ_0 but $V \to C$ is an FD in Σ_0. Based on this FD we construct a relation with ℓ_2 tuples $t'_1, \ldots, t'_{\ell_2}$ that all have matching values on columns in $V^+_{\Sigma_0[\text{FD}]} = VC$, and unique values on all other columns. The relation may consist of the last ℓ_2 tuples in r of Table 1a. The relation exemplifies the stronger fact that ℓ_2 is the equilibrium between update inefficiency and join efficiency exhibited by (Hap, Σ_0).

For our running example we have arrived at an instance of the fundamental question whether (Hap, Σ_0) can be transformed into a schema that is in ℓ-BCNF with $\ell < \ell_2$.

5 Schema Design Algorithms

After defining the update inefficiency and join efficiency of schema decompositions we propose three algorithms for logical schema design. We also illustrate how our concepts inform view definition and selection, already during logical design.

5.1 Assessing Decompositions

We can assess the quality of a schema decomposition $\mathcal{D}$ for both updates and joins.

Join efficiency. The join efficiency of $\mathcal{D}$ results from only those FDs preserved by $\mathcal{D}$. These may have been transformed into keys or not. We define the set of *join-supportive attribute subsets* as

$$\begin{aligned} JS^{(R,\Sigma)}_{\mathcal{D}} \quad = \quad & \{S : X_k \mid \exists S \in \mathcal{D} \exists X \to Y \in \Sigma[S], \Sigma \models X \to S\} \cup \\ & \{S : X \mid \exists S \in \mathcal{D} \exists X \to Y \in \Sigma[S], \Sigma \not\models X \to S\} \ . \end{aligned}$$

Next, the join strength of a join-supportive attribute set is given by the minimal upper bounds for any CC that applies to it. Hence, for (R, Σ) and $X \subseteq R$, $\ell_X := \min\{\ell \in \mathbb{N}^{\infty}_{\geq 1} \mid \Sigma \models card(X) \leq \ell\}$ is the minimum level of data redundancy for X. The level of join efficiency for $\mathcal{D}$ is the maximum among all minimum levels of data redundancy for a join-supportive attribute set, that is,

$$\ell^J_{\mathcal{D}} := \max\{\ell_X \mid S : X \in JS^{(R,\Sigma)}_{\mathcal{D}}\} \cup \{1 \mid S : X_k \in JS^{(R,\Sigma)}_{\mathcal{D}}\}.$$

FDs $X \to Y$ transformed into keys X contribute level 1, and other FDs contribute level ℓ_X. Note that $\ell^J_{\mathcal{D}}$ is 1 when Σ contains no FDs. The *total level of join efficiency for* $\mathcal{D}$ is the sum of the levels:

$$\ell^{J,\text{total}}_{\mathcal{D}} = \sum_{S : X \in JS^{(R,\Sigma)}_{\mathcal{D}}} \ell_X + \sum_{S : X_k \in JS^{(R,\Sigma)}_{\mathcal{D}}} 1.$$

In our case study for (Hap, Σ_0) from Example 3.2, decomposition $\mathcal{D}_g = \{(R_2 = CV, \Sigma_2 = \{V \to C\}), (R_3 = ETV, \Sigma_3 = \{TV \to E, ET \to V\})\}$ leads to the join-supportive attribute sets $R_2 : V_k$, $R_3 : (TV)_k$ and $R_3 : (ET)_k$ with $\ell^J_{\mathcal{D}_g} = 1$ and $\ell^{J,\text{total}}_{\mathcal{D}_g} = 3$. For the decomposition $\mathcal{D}_1 = \{(R_2, \Sigma_2), (R_3, \Sigma_3), (R_4, \Sigma_4)\}$ we have the join-supportive attribute sets $R_2 : V_k$, $R_3 : (TV)_k$, $R_3 : (ET)_k$, $R_4 : (CT)_k$ and $R_4 : E$ with $\ell_E = 1k$. Hence, we have $\ell^J_{\mathcal{D}_1} = 1,000$ and $\ell^{J,\text{total}}_{\mathcal{D}_1} = 1,004$.

Update inefficiency. The update inefficiency of $\mathcal{D}$ is determined by all FDs of the input. In particular, any lost FD will need to be enforced by joining elements of $\mathcal{D}$ whenever updates occur. A BCNF decomposition of our running example into R_2 and R_3 results in a loss of the FD $E \rightarrow C$. As the instances r_2 and r_3 in Table 1b illustrate, any update of a C-value (such as *Kilo*) in r_2 needs to be propagated consistently for all its occurrences in r_2 with the same event (eg. *Party*), so the FD $E \rightarrow C$ still holds (on the join of R_2 and R_3). The example illustrates how lost FDs cause data redundancy on the join of schemata. This is the reason why they need to be enforced. Hence, we define the set of *update-critical attribute subsets* as

$$UC_{\mathcal{D}}^{(R,\Sigma)} = JS_{\mathcal{D}}^{(R,\Sigma)} \cup \{R : X \mid \exists X \rightarrow Y \in (\Sigma - (\cup_{S \in \mathcal{D}} \Sigma[S])^+)\}.$$

Now we define the level of update inefficiency for $\mathcal{D}$ as the maximum among all minimum levels of data redundancy for an update-critical attribute set, that is,

$$\ell_{\mathcal{D}}^{U} := \max\{\ell_X \mid S : X \in UC_{\mathcal{D}}^{(R,\Sigma)}\} \cup \{1 \mid S : X_k \in UC_{\mathcal{D}}^{(R,\Sigma)}\}.$$

Alternatively, we can also sum up to derive the *total level of update inefficiency for* $\mathcal{D}$ as follows:

$$\ell_{\mathcal{D}}^{U,\text{total}} = \sum_{S:X \in UC_{\mathcal{D}}^{(R,\Sigma)}} \ell_X + \sum_{S:X_k \in UC_{\mathcal{D}}^{(R,\Sigma)}} 1.$$

Continuing our example, decomposition $\mathcal{D}_g$ leads to additional update-critical attribute subsets due to losing $CT \rightarrow V$ and $E \rightarrow C$ from the input set Σ_0: $R : CT$ and $R : E$ with $\ell_{CT} = 1$ and $\ell_E = 1k$. Hence, $\ell_{\mathcal{D}_g}^{UC} = 1,000$ and $\ell_{\mathcal{D}_g}^{UC,\text{total}} = 1,004$. Since the decomposition $\mathcal{D}_1$ is FD-preserving, the levels of update inefficiency coincide with the levels of join efficiency from before, respectively.

Overcoming limitation of classical normalization. With only FDs, BCNF decomposition or 3NF synthesis may lead to the optimal case of an FD-preserving BCNF decomposition where the level of update inefficiency is 1. Otherwise, some non-trivial FD is lost or not a key. Without CCs, every non-optimal case results in unbounded levels of update inefficiency and join efficiency. With CCs, our framework can measure update inefficiency and join efficiency for every schema. We will now propose algorithms for schema design, and illustrate their achievements later by experiments.

5.2 FD-preservation

FD preservation is fundamental to the ability of maintaining data consistency on individual tables. In fact, whenever updates occur that potentially violates some lost FD, a join of some tables is required to validate the FD. As this may be prohibitively expensive, FD preservation is an important goal. This goal has been addressed previously by synthesis into 3NF. However, with FDs alone, any schema that is in 3NF but not in BCNF will exhibit levels of update inefficiency and join efficiency that are a priori unbounded. In our new framework, we therefore aim at minimizing the level of update inefficiency across all FD-preserving decompositions. In addition, preserving all FDs guarantees that the levels of join efficiency and update inefficiency coincide. Otherwise, the level of update inefficiency may exceed that of join efficiency (more pain than gain).

Based on previous work by Osborne [52], we utilize the unique *atomic closure* Σ_a that consists of all L-reduced FDs with singleton attribute on the right-hand side for a given set Σ. Indeed, Osborne proved that a schema (R, Σ) with FD set Σ has a lossless, FD-preserving BCNF decomposition if and only if there is an atomic cover of Σ_a that does not exhibit any *critical* FD, that is, an FD $X \rightarrow A$ such that $(XA, \Sigma^+[XA])$ is not in BCNF [52].

Algorithm 2 starts with atomic closure Σ_a of Σ (line 1). It then checks for all $X \rightarrow A$ in Σ_a (line 2) whether they are critical (line 4), that is whether there is some non-trivial FD $Y \rightarrow B \in \Sigma_a$ whose attributes are contained in XA, but where Y is not a key for XA. In such a case, we compute the minimum level ℓ_Y associated with $Y \rightarrow B$ (line 5) and keep track of the FD $Y \rightarrow B$ and ℓ_Y

ALGORITHM 2: Opt(R, Σ)

Require: (R, Σ) with CC/FD set Σ over schema R
Ensure: Lossless, FD-preserving $\ell_{\mathcal{D}}$-BCNF decomposition $\mathcal{D}$ of (R, Σ) with best $\ell_{\mathcal{D}}$ $(= \ell^U_{\mathcal{D}} = \ell^J_{\mathcal{D}})$
1: Compute the atomic closure Σ_a of Σ [31, 52];
2: **for all** $X \to A \in \Sigma_a$ **do**
3: $\quad \Sigma[X \to A] \leftarrow \emptyset$
4: $\quad$ **for all** $Y \to B \in \Sigma_a (YB \subseteq XA \wedge XA \not\subseteq Y^+_{\Sigma[FD]})$ **do**
5: $\quad\quad \ell_Y \leftarrow \min\{\ell \mid \Sigma \models card(Y) \leq \ell\}$
6: $\quad\quad \Sigma[X \to A] \leftarrow \Sigma[X \to A] \cup \{(Y \to B, \ell_Y)\}$
7: $\quad \ell_{X \to A} \leftarrow \sup\{\ell_Y \mid Y \to B \in \Sigma[X \to A]\}$ //$\ell_{X \to A} = 1$, if $X \to A$ is not critical
8: $\mathcal{D} \leftarrow \emptyset$;
9: **for all** $X \to A \in \Sigma_a$ in decreasing order of $\ell_{X \to A}$ **do**
10: $\quad$ **if** $\Sigma_a - \{X \to A\} \models X \to A$ **then**
11: $\quad\quad \Sigma_a \leftarrow \Sigma_a - \{X \to A\}$
12: $\quad$ **else**
13: $\quad\quad \mathcal{D} \leftarrow \mathcal{D} \cup \{(XA, \Sigma_a[XA])\}$
14: Remove all $(S, \Sigma_a[S]) \in \mathcal{D}$ if $\exists (S', \Sigma_a[S']) \in \mathcal{D} (S \subseteq S')$
15: **if** there is no $(R', \Sigma') \in \mathcal{D}$ where $R' \to R \in \Sigma^+_{\mathcal{Q}}$ **then**
16: $\quad$ Choose a minimal key K for R with respect to Σ
17: $\quad \mathcal{D} \leftarrow \mathcal{D} \cup \{(K, \Sigma_a[K])\}$
18: **Return**$(\mathcal{D}, \ell_{\mathcal{D}})$

(line 6). Subsequently, we compute for $X \to A$ the maximum $\ell_{X \to A}$ that is associated with all the non-key FDs subsumed by $X \to A$ (line 7). In particular, if $X \to A$ is not critical, $\ell_{X \to A} = 1$ (line 7). Subsequently (from line 8 onwards), we synthesize the final decomposition with schemata generated by FDs σ in decreasing order of their ℓ_σ (line 9). In particular, if any FD is redundant (line 10) we remove it (line 11). This ensures that we can remove critical FDs that cause the highest levels of data redundancy first. Consequently, there cannot be any lossless FD-preserving decomposition into ℓ-BCNF with smaller ℓ. However, if an FD is not redundant (line 12), we will add it to the decomposition (line 13). Similar to 3NF synthesis, we remove schemata who are contained in other schemata in the decomposition (line 14), and we ensure to add a minimal key (line 16/17) whenever the current decomposition does not already imply a key (line 15). This step is necessary to ensure losslessness since an FD-preserving decomposition is lossless if and only if it contains a minimal key [7]. Due to Theorem 3.7, Algorithm 2 is worst-case exponential.

Theorem 5.1. *Given (R, Σ), Algorithm 2 computes a lossless, FD-preserving decomposition into ℓ-BCNF with the minimum ℓ possible among all lossless, FD-preserving decompositions.*

(Hap, Σ_0) from Example 3.2 is already in 3NF. As no FD-preserving BCNF decomposition exists, classical normalization stops here. Using (Hap, Σ_0) as input for Algorithm 2, we obtain $\Sigma_a = \Sigma_0 \cup \{CT \to E\}$ (line 1). Hence, $\ell_{CT \to V} = 1m$ and $\ell_{CT \to E} = 1k$, and for other $X \to A \in \Sigma_a$, $\ell_{X \to A} = 1$ (line 7). However, $CT \to V$ is implied by $\Sigma_a - \{CT \to V\}$ (line 10). This step is essential, since $CT \to V$ has effectively been replaced by $CT \to E$, which also means that the *1m*-level data redundancy caused by $CT \to V$ has been reduced to the *1k*-level data redundancy caused by $CT \to E$. Furthermore, $R_1 \subseteq R_4$ (line 14), so Algorithm 2 returns the 1*k*-BCNF schema $\mathcal{D}_1 = \{(R_2, \Sigma_2), (R_3, \Sigma_3), (R_4, \Sigma_4)\}$ from the intro. Classical normalization would generate neither $\mathcal{D}_1$ nor $\mathcal{D}_2$. Even if it did, $\mathcal{D}_1$, $\mathcal{D}_2$ and (Hap, Σ_0) would be assessed as equal in quality.

ALGORITHM 3: GREED(R, Σ)

Require: (R, Σ) with CC/FD set Σ over R, $\ell \in \mathbb{N}_{\geq 1}$
Ensure: Lossless $\ell^U_{\mathcal{D}}$-BCNF decomposition $\mathcal{D}$ of (R, Σ)
1: **if** (R, Σ) is in ℓ-BCNF **then**
2: $\quad \mathcal{D} \leftarrow \{(R, \Sigma)\}$
3: **else**
4: $\quad \ell_{\max} \leftarrow \max\{\ell_X \mid X \rightarrow Y \in \Sigma, Y \not\subseteq X, \Sigma \not\models X \rightarrow R\}$
5: $\quad$ Pick $X \rightarrow Y \in \Sigma$ $(Y \not\subseteq X \wedge \Sigma \not\models X \rightarrow R \wedge \ell_X = \ell_{\max})$
6: $\quad R_1 \leftarrow XY$; $R_2 \leftarrow X(R - XY)$
7: $\quad \mathcal{D} \leftarrow$ GREED$(R_1, \Sigma[R_1]) \cup$ GREED$(R_2, \Sigma[R_2])$
8: **Return**$(\mathcal{D}, \ell^U_{\mathcal{D}}, \ell^J_{\mathcal{D}})$

We may apply Algorithm 2 to a partial input of Σ. For example, excluding FDs, which potentially cause high update inefficiency but are unlikely violated by actual updates, may result in higher join efficiency. Similarly, one may include all FDs $X \rightarrow Y$ where ℓ_X meets a given threshold.

5.3 Greedy Decomposition

Typically, FD-preservation requires many output tables, which is linked to low join efficiency. We reposition classical BCNF decomposition as a greedy algorithm for reducing $\ell^U_{\mathcal{D}}$ using fewer tables.

Selecting non-key FDs that drive classical BCNF decomposition is arbitrary. With CCs at hand, we can select FDs whose level of data redundancy is maximal (for example, ∞). Algorithm 3 shows this strategy. Targeting higher join efficiency, we may use a higher level ℓ as input. If unspecified, the default target is $\ell = 1$. The algorithm proceeds with an FD that exhibits a maximum level of data redundancy among those on the schema (lines 4–5), as long as some local schema is not in ℓ-BCNF (line 1). Note that the ℓ in line 1 is only guaranteed in terms of the FDs that have been preserved, but not in terms of FDs that have been lost. Hence, the output returns schema $\mathcal{D}$, the level $\ell^U_{\mathcal{D}}$ of update inefficiency for which it is effective, and the level $\ell^J_{\mathcal{D}}$ of join efficiency it permits.

The penalty for fewer output tables is the lack of control over the levels of update inefficiency and join efficiency, which is due to lost FDs. Note that FDs $X \rightarrow A$ where $\ell_X = \infty$ have high priority. Hence, if no bound is specified, those FDs are prioritized. With no CCs given at all, Algorithm 3 reduces to classical BCNF decomposition where $\ell^U_{\mathcal{D}} = \infty = \ell^J_{\mathcal{D}}$, unless the output $\mathcal{D}$ is FD-preserving, in which case $\ell^U_{\mathcal{D}} = 1 = \ell^J_{\mathcal{D}}$.

We apply Algorithm 3 to (HAP, Σ_0) from Example 3.2. As $V \rightarrow C \in \Sigma_0$ causes $\ell_{\max} = 1m$ (line 4), we obtain $\mathcal{D}_g = \{(R_2 = VC, \Sigma_2 = \{V \rightarrow C\}), (R_3 = EVT, \Sigma_3 = \{TV \rightarrow E, ET \rightarrow V\})\}$ (line 7). We lost $E \rightarrow C$ and $CT \rightarrow V$, so $\ell^U_{\mathcal{D}_g} = 1k$. Lost FDs can be addressed by creating assertion checks. For instance, we can create such checks for the lost FDs $E \rightarrow C$ and $CT \rightarrow V$ on the join $R_2 \bowtie R_3$:

```
CREATE ASSERTION LostFD_E_to_C
CHECK( NOT EXISTS (
SELECT R3.E
FROM R2, R3
WHERE R2.V = R3.V
GROUP BY R3.E
HAVING COUNT(R2.C)>1));
```

```
CREATE ASSERTION LostFD_CT_to_V
CHECK( NOT EXISTS (
SELECT R2.C, R3.T
FROM R2, R3
WHERE R2.V = R3.V
GROUP BY R2.C, R3.T
HAVING COUNT(R2.V)>1));
```

$\mathcal{D}_g$ achieves the same level of update inefficiency as $\mathcal{D}_1$ with fewer schemata. This comes at the price of lost FDs that require assertion checks and smaller join efficiency $\ell^J_{\mathcal{D}_g} = 1$.

ALGORITHM 4: HYBRID(R, Σ)

Require: (R, Σ) with CC/FD set Σ over schema R
Ensure: Lossless $\ell^U_{\mathcal{D}}$-BCNF decomposition $\mathcal{D}$ of (R, Σ)
1: $\mathcal{D} \leftarrow$ OPT(R, Σ)
2: **for all** $S \in \mathcal{D}$ **do**
3: $S = XA$ for some $X \rightarrow A \in \Sigma_a$
4: **for all** $Y \rightarrow B \in \Sigma_a (YB \subseteq XA \wedge XA \not\subseteq Y^+_{\Sigma[FD]})$ **do**
5: **if** $\ell_Y > \ell_X$ **then**
6: $S_1 = YB$; $S_2 = Y(XA - B)$
7: $\mathcal{D} \leftarrow (\mathcal{D} - \{(S, \Sigma_a[S])\}) \cup \{(S_1, \Sigma_a[S_1]), (S_2, \Sigma_a[S_2])\}$
8: Remove any non-maximal schema from $\mathcal{D}$
9: **Return**$(\mathcal{D}, \ell^U_{\mathcal{D}}, \ell^J_{\mathcal{D}})$

Our algorithms can be adapted as more information becomes available. For example, while $\mathcal{D}_g$ was driven by $V \rightarrow C$ due to $card(V) \leq 1m$, there may not be updates of C-values based on V-values but frequent updates of C-values based on E-values. Hence, we could prioritize $E \rightarrow C$ instead to obtain $\mathcal{D}'_g = \{(R_1 = EC, \Sigma_1 = \{E \rightarrow C\}), (R_3, \Sigma_3)\}$ with lost FDs $V \rightarrow C$ and $CT \rightarrow V$.

Just like lossless, FD-preserving BCNF decompositions cannot always be achieved, lossless, FD-preserving decompositions into ℓ-BCNF cannot always be achieved, for any given ℓ.

PROPOSITION 5.2. *For every positive integer ℓ there is a schema (R, Σ) for which no lossless, FD-preserving decomposition into ℓ-Bounded Cardinality Normal Form exists.*

5.4 Hybrid Algorithm

We combine the two previous strategies to the hybrid Algorithm 4. Here, Algorithm 2 is applied first (line 1) to obtain the smallest possible $\ell_{\mathcal{D}}$ among all FD-preserving decompositions. Next we check if for any of the output schemata XA (lines 2-3) that are in 3NF but not in BCNF (line 4), any non-key FD $Y \rightarrow B$ on XA satisfies $\ell_Y > \ell_X$ (line 5). In this case, we trade the preservation of $X \rightarrow A$ for the lower level ℓ_X of update inefficiency by decomposing with the FD $Y \rightarrow B$ (lines 6/7).

On input (HAP, Σ_0) from Example 3.2, Algorithm 4 returns decomposition $\mathcal{D}_1$ as output of Algorithm 2 (line 1). In particular, $S = R_4 = CET$ is in 3NF but not in BCNF for $\Sigma_a[S] = \Sigma_4 = \{CT \rightarrow E, E \rightarrow C\}$. That is, the FD $E \rightarrow C \in \Sigma_a$ causes level ℓ_E data redundancy on the schema $S = R_4 = CET$ (lines 3 and 4). Since $\ell_E = 1k > 1 = \ell_{CT}$ (line 5), we decompose along the FD $E \rightarrow C$ to replace R_4 by $R_6 = EC$ and $R_7 = ET$ (lines 6 and 7). However, $R_6 = R_1$ and $R_7 \subseteq R_3$. Hence, the output is $\mathcal{D}_h = \{(R_1, \Sigma_1), (R_2, \Sigma_2), (R_3, \Sigma_3)\}$ with $\ell^U_{\mathcal{D}_h} = 1 = \ell^J_{\mathcal{D}_h}$ and lost FD $CT \rightarrow V$. The assertion check `LostFD_CT_to_V` enforces the FD $CT \rightarrow V$ on the join $R_2 \bowtie R_3$.

Table 5 summarizes the schemata our algorithms return and also $\mathcal{D}_2$, including their properties.

5.5 Materialized Views

Once a database becomes operational, frequent access patterns emerge over time. Adding materialized views can help accelerate queries but will occupy additional storage space and time to maintain data consistency. Hence, the definition and selection of views should be informed by quantifying the support of joins and the costs associated with maintaining data consistency. We define the (total) level of join efficiency and update inefficiency of a given view $\mathcal{V}$ (namely, a set of attributes) as $\ell^{J(,\text{ total})}_{\mathcal{V}}$ and $\ell^{U(,\text{ total})}_{\mathcal{V}}$, respectively.

For our hybrid schema $\mathcal{D}_h$ as example, a frequent query may ask at what times companies work on a venue. Hence, we may introduce the following materialized view $\mathcal{V}$.

Table 5. Schema Designs for Running Example

Method	*Schema*	*Lost FDs*	ℓ^U	ℓ^J	Materialized View
Opt	$\mathcal{D}_1$: $R_2 = CV$ with $V \rightarrow C$ $R_3 = ETV$ with $TV \rightarrow E, ET \rightarrow V$ $R_4 = CET$ with $CT \rightarrow E, E \rightarrow C$	none	1k	1k	$\mathcal{V}_{\mathcal{D}_1} = \pi_{VCT}(R_2 \bowtie R_3)$
Synthesis	$\mathcal{D}_2$: $R_1 = EC$ with $E \rightarrow C$ $R_3 = ETV$ with $TV \rightarrow E, ET \rightarrow V$ $R_5 = CTV$ with $CT \rightarrow V, V \rightarrow C$	none	1m	1m	$\mathcal{V}_{\mathcal{D}_2} = \pi_{ECT}(R_1 \bowtie R_3)$
Greed	$\mathcal{D}_g$: $R_2 = CV$ with $V \rightarrow C$ $R_3 = ETV$ with $TV \rightarrow E, ET \rightarrow V$	$E \rightarrow C$ $CT \rightarrow V$	1k	1	$\mathcal{V}_{\mathcal{D}_g} = \pi_{ECT}(R_2 \bowtie R_3)$
Hybrid	$\mathcal{D}_h$: $R_1 = EC$ with $E \rightarrow C$ $R_2 = CV$ with $V \rightarrow C$ $R_3 = ETV$ with $TV \rightarrow E, ET \rightarrow V$	$CT \rightarrow V$	1	1	$\mathcal{V}_{\mathcal{D}_h} = \pi_{VCT}(R_1 \bowtie R_3)$

CREATE MATERIALIZED VIEW $\mathcal{V}$ AS (
SELECT $R_2.V, R_2.C, R_3.T$
FROM R_2, R_3
WHERE $R_2.V = R_3.V$);

In this case, the assertion check `LostFD_CT_to_V` can directly be enforced on the view instead.

CREATE ASSERTION LostFD_CT_to_V CHECK(NOT EXISTS(
SELECT C, T
FROM $\mathcal{V}$
GROUP BY C, T
HAVING COUNT(V)>1));

Due to $V \rightarrow C$, $CT \rightarrow V$ and $card(V) \leq 1m$, the view has level $\ell^J_{\mathcal{V}} = 1m = \ell^U_{\mathcal{V}}$ update inefficiency and join efficiency, respectively. Join support is effective, but requires the propagation of a C-update on base table R_2 to up to $1m$ updates on $\mathcal{V}$. Hence, the equilibrium we quantify between the smallest level of update inefficiency and the largest level of join efficiency is also intrinsic to materialized views. It should therefore inform the definition and selection of materialized views. Table 5 also lists some views for the various schema designs of our running example.

5.6 Degrees of CARE for Database Normalization

Our results amplify the intrinsic tradeoff between update and query efficiency, and therefore the importance of designing and normalizing database schemata carefully. Our work promotes the use of CCs in the database design process as it brings forward an opportunity to measure worst-case levels of data redundancy and best-case levels of join strengths, quantifying the tradeoff between update and query efficiency. The following guidelines stipulate degrees of CARE that one may apply during database normalization.

(1) Degree of **C**ybernization: As with any algorithm, the output should only be perceived as a recommendation, to be assessed by human experts before making a decision. In practice, Algorithm 2 guarantees a lossless, FD-preserving decomposition into 3NF, potentially in BCNF. However, for a large number of input FDs, the output may consist of a large number of schemata, may be over-normalized, and impractical for future workloads. Indeed, it will be critically important to carefully think about the input to any database normalization algorithm, in addition to carefully evaluating the output.

(2) Degree of **A**daptation: Further normalization, de-normalization and the use of auxiliary data structures such as views and integrity checks can address some needs for more efficient query and update support. This can be applied when requirements have not been captured well, or when these change. Nevertheless, our work has amplified the tradeoff between update and query efficiency, without and with views, so support for both kinds of operations at the same time will always be limited. Keeping up do date with application requirements remains a major concern in carefully adapting the underlying database schema accordingly.
(3) Degree of **R**egularization: Put boldly, we neither want to over-normalize nor under-normalize, whatever our means are. With many FDs as input, we can always find a quality lossless, FD-preserving decomposition into 3NF. If the number of output schemata is too high, we may have over-normalized and we should further prioritize FDs for input. If we have only few FDs as input, the greedy algorithm will only generate a small number of schemata, perhaps with no loss of FDs. However, we should watch out for under-normalization in this case. The hybrid algorithm may be of use when some schemata in 3NF still permit high levels of data redundancy that are subject to frequent updates. Again, a careful consideration for the input to normalization algorithms is of utmost importance.
(4) Degree of **E**licitation: As emphasized in our previous points, the degree of requirements elicitation is critical for the suitability of database schemata that result from normalization algorithms. Our best advice is to dedicate sufficient resources to requirements elicitation. The most important question that needs continued attention is:

    Which FDs will cause (many) redundant data values that are subject to (frequent) updates?

    CCs help pinpoint worst-case levels of data redundancy, but analyzing how often redundant values on such columns may actually occur will provide more insight how important the underlying FD for normalization is. Likewise, we may base our priority of an FD on the frequency of updates, and this may be of particular importance if this FD is not a key dependency on a schema in 3NF. Our results suggest to invest resources in requirements elicitation, particularly to fill gaps in understanding how many redundant data values may be associated with FDs, and how frequent updates and queries involving these values will be.

Overall, these are good news that put forward specific focus points for requirements elicitation. The news are even better when normalization is applied as a further optimization step after the conceptual database design phase [61, 65]. Commonly, it will be rare that redundant values require updates while also being needed for answering queries, particularly for applications based on reads rather than writes, or vice versa. The former applications may favor more de-normalization while the latter may favor less. We will always aim for normalizing at the right degree and organize data into semantically meaningful units. Indeed, queries benefit from normalization when joins of small intermediate results outperform selects from huge tables. Finally, we point out that both

— the tradeoff between update and join efficiency quantified in this article, and
— the benefits of (de-)normalization

intrinsically apply to every model of data, with different aspects magnified differently.

6 Experiments

In Sections 6.1 and 6.2, our experiments analyze how the properties of our normal forms translate into the performance of updates and joins over instances of the schema. In Section 6.3, we show how the levels of data redundancy and number of redundant data value occurrences can inform the recommendation of dimension tables in automated star schema designs. Our algorithms are

Table 6. Queries q_1 and q_2 on Schema Designs for Running Example

q	$\mathcal{D}_1$	$\mathcal{D}_1 + \mathcal{V}_{\mathcal{D}_1}$
q_1	R_4	R_4
q_2	$\pi_{CTV}(R_2 \bowtie R_3)$	$\mathcal{V}_{\mathcal{D}_1}$

(a) Decomposition $\mathcal{D}_1$

q	$\mathcal{D}_2$	$\mathcal{D}_2 + \mathcal{V}_{\mathcal{D}_2}$
q_1	$\pi_{ECT}(R_1 \bowtie R_3)$	$\mathcal{V}_{\mathcal{D}_2}$
q_2	R_5	R_5

(b) Decomposition $\mathcal{D}_2$

q	$\mathcal{D}_g$	$\mathcal{D}_g + \mathcal{V}_{\mathcal{D}_g}$
q_1	$\pi_{ECT}(R_2 \bowtie R_3)$	$\mathcal{V}_{\mathcal{D}_g}$
q_2	$\pi_{CTV}(R_2 \bowtie R_3)$	$\pi_{CTV}(R_2 \bowtie R_3)$

(c) Decomposition $\mathcal{D}_g$

q	$\mathcal{D}_h$	$\mathcal{D}_h + \mathcal{V}_{\mathcal{D}_h}$
q_1	$\pi_{ECT}(R_1 \bowtie R_3)$	$\pi_{ECT}(R_1 \bowtie R_3)$
q_2	$\pi_{CTV}(R_1 \bowtie R_3)$	$\mathcal{V}_{\mathcal{D}_h}$

(d) Decomposition $\mathcal{D}_h$

Table 7. Synthetic Experiment 1 (in Seconds)

Op	Hap	$\mathcal{D}_1$	$\mathcal{D}_1 + \mathcal{V}_{\mathcal{D}_1}$	$\mathcal{D}_2$	$\mathcal{D}_2 + \mathcal{V}_{\mathcal{D}_2}$	$\mathcal{D}_g$	$\mathcal{D}_g + \mathcal{V}_{\mathcal{D}_g}$	$\mathcal{D}_h$	$\mathcal{D}_h + \mathcal{V}_{\mathcal{D}_h}$
u_1	0.93	0.91	0.91	0.59	1.03	1.62	1.81	0.59	0.59
u_2	45.75	0.23	45.33	44.81	44.81	0.20	0.20	0.21	83.82
q_1	0.011	0.009	0.009	0.17	0.012	1.32	0.009	0.14	0.14
q_2	3.39	4.11	3.47	2.76	2.76	4.09	4.09	4.32	3.44

implemented in Visual C++. Experiments were done on an Intel Xeon W-2123, 3.6 GHz, 256GB, Windows 10 PC, with the 2017 SQL Server Community Edition.

6.1 Qualitative Study

Firstly, we analyze the performance of two updates and two joins on synthetic instances over the schema designs for our running example.

Our first instance over (Hap, Σ_0) consists of 1,001,000 tuples. There are $1k$ tuples with matching values on E and C, and $1m$ tuples with matching values on V and C. The other values are unique in their columns. The instance is isomorphic to instance r in Table 1a with $\ell_1 = 1k$ and $\ell_2 = 1m$. To illustrate the impact of FDs on updates and joins, the numbers of redundant values they cause in the instance coincide with their levels of data redundancy on the schema (ℓ_1 and ℓ_2, respectively).

For the experiments, we consider four operations that are affected by our CCs and FDs:

— Update u_1 replaces all occurrences of a C-value associated with a given E-value,
— Update u_2 replaces all occurrences of a C-value associated with a given V-value,
— Query q_1 returns all CTE combinations, and
— Query q_2 returns all CTV combinations.

Each operation is run 100 times on each schema design in Table 5, and the average value reported.

For u_1, $1k$ occurrences of a redundant C-value are updated, and $1m$ occurrences of a redundant C-value are updated for u_2, due to the non-key FDs $E \rightarrow C$ and $V \rightarrow C$, respectively. Updates on base tables are always propagated to materialized views, if those are present. As $E \rightarrow C$ is lost on $\mathcal{D}_g$, u_1 updates C-values on $R_2 \bowtie R_3$ and projects them onto R_2.

Table 6 shows how the queries q_1 and q_2 are implemented on each of our schemata from Table 5, without and with materialized views. On the instance over 3NF schema (Hap, Σ), q_1 and q_2 are simple projections and do not require joins.

Table 7 shows times (in sec) for operations on the instance projected on the schemata of Table 5.

Table 8. Synthetic Experiment 2 (in Milliseconds)

Op	Hap	$\mathcal{D}_1$	$\mathcal{D}_1 + \mathcal{V}_{\mathcal{D}_1}$	$\mathcal{D}_2$	$\mathcal{D}_2 + \mathcal{V}_{\mathcal{D}_2}$	$\mathcal{D}_g$	$\mathcal{D}_g + \mathcal{V}_{\mathcal{D}_g}$	$\mathcal{D}_h$	$\mathcal{D}_h + \mathcal{V}_{\mathcal{D}_h}$
u_1	58.2	55.8	55.8	3.7	60.7	82.3	170.2	3.7	3.7
u_2	10	4.4	12.7	9.8	9.8	4.9	4.9	4.7	18.9
q_1	8.3	7.5	7.5	11.1	10.6	236	4	11.6	11.6
q_2	5.6	14.6	5.7	5.3	5.3	14.5	14.5	14.1	7.4

Specific updates and queries are best supported by different designs. Notably, the level of data redundancy for an FD translates proportionally into the performance for updates and joins affected by the FD. This also applies to the materialized views.

Operations u_2 and q_1 are fast on $\mathcal{D}_1$ with R_4 and key V on R_2, but slow on $\mathcal{D}_2$. Symmetrically, u_1 and q_2 are fast on $\mathcal{D}_2$ with R_5 and key E on R_1, but slow on $\mathcal{D}_1$. For each operation, the difference in performance between $\mathcal{D}_1$ and $\mathcal{D}_2$ is proportional to the level of data redundancy caused by the FD that affects the operation. For example, the FD $V \rightarrow C$ with $\ell_V = 1m$ causes a significant difference between performing u_2 (44.58s difference) on $\mathcal{D}_1$ and $\mathcal{D}_2$, and between performing q_2 (1.35s difference) on $\mathcal{D}_1$ and $\mathcal{D}_2$. In comparison, the FD $E \rightarrow C$ with $\ell_V = 1k$ causes a much smaller difference between u_1 (0.32s) on $\mathcal{D}_1$ and $\mathcal{D}_2$, and between q_1 (0.161s) on $\mathcal{D}_1$ and $\mathcal{D}_2$.

Relatively to $\ell_{\mathcal{V}_{\mathcal{D}_1}} = 1m$, the speed up of q_2 and slow down of u_2 by $\mathcal{V}_{\mathcal{D}_1}$ are large. Relatively to $\ell_{\mathcal{V}_{\mathcal{D}_2}} = 1k$, the speed up of q_1 and slow down of u_1 by $\mathcal{V}_{\mathcal{D}_2}$ are smaller.

As expected, $\mathcal{D}_g$ performs well on u_2 but not so much on the other operations. $\mathcal{V}_{\mathcal{D}_g}$ speeds up q_1 with small penalty for u_1, but which had poor performance already.

$\mathcal{D}_h$ does well on u_1 and u_2, and quite well on q_1. $\mathcal{V}_{\mathcal{D}_h}$ speeds up q_2 but slows down u_2 significantly.

Table 8 shows our results for the second synthetic instance with 1,100 tuples, $\ell_1 = 1k$, and $\ell_2' = 100 < 1m$. The experiment illustrates that the same combinations of operations are best supported by the same designs, when compared to the first instance. Since the actual cardinality (100) is much smaller than what is permitted ($1m$), processing times for operations with this cardinality are much smaller (in ms).

The choice of a logical design depends on how well it is believed to support the future workload of operations. Our framework quantifies this support by the level of data redundancy that FDs exhibit as part of the design or materialized view, which measures their impact on the performance of specific updates and joins. In contrast, classical logical design may not bring forward some designs and cannot quantify the support for workloads. For example, schema (Hap, $\overline{\Sigma}_0$) from Example 3.2 is in 3NF and no lossless, FD-preserving decomposition into BCNF exists. Hence, classical design based on FDs alone would stop there. In contrast, by including CCs in the analysis, our methods transform the schema (Hap, Σ_0) from Example 3.2 into the various schema designs listed in Table 5 and quantify the achievements in terms of update inefficiency and join efficiency.

6.2 Quantitative Study

Secondly, we analyze update and join performance over schema designs derived by our algorithms for FDs and CCs we mined from the real-world datasets in Table 9. It shows the number of FDs discovered (#FD), the total number of null marker occurrences (#⊥), and the number of rows and columns (#R,#C). Here, #FD refers to the number of FDs in the atomic closure, consisting of the set of LHS minimal FDs $X \rightarrow A$ with singleton attribute A that hold on the dataset.

The datasets benchmark algorithms that discover FDs $X \rightarrow A$ from data and rank them in decreasing order by the lowest bound ℓ_X such that $card(X) \leq \ell_X$ holds on the data [53, 54, 67]. We regard different occurrences of ⊥ in the same column as matching domain values (⊥=⊥). Our

Table 9. Datasets Used in Experiments

Data set	#FDs	#⊥	#R	#C
china	918	418580	262920	18
ncvoter	568	3079822	1024000	19

Table 10. Algorithmic Achievements on Real Schemata

measures	Original	Opt	Greed	Hybrid
ℓ^U	4, 097	15	512	7
ℓ^J	4, 097	15	1	7
$\ell^{U,\text{total}}$	5, 043	87	3, 035	79
$\ell^{J,\text{total}}$	5, 043	87	6	73
$\lvert\mathcal{D}\rvert$	1	65	6	66
time (ms)	-	< 1	< 1	< 1

(a) Measures for top-20% FDs of **ncvoter**

measures	Original	Opt	Greed	Hybrid
ℓ^U	4, 889	56	4, 881	55
ℓ^J	4, 889	56	1	50
$\ell^{U,\text{total}}$	62, 492	327	54, 247	257
$\ell^{J,\text{total}}$	62, 492	327	4	165
$\lvert\mathcal{D}\rvert$	1	145	4	157
time (ms)	-	< 1	< 1	< 1

(b) Measures for top-20% FDs of **China**

results are very similar otherwise ($\perp \neq \perp$). The discovered CCs and FDs represent patterns on the data, but not necessarily on the domain. Input to our algorithms are only more relevant FDs, namely those ranked in the top-20%. Adding further FDs causes hardly any differences, indicating that FDs with higher levels of data redundancy determine schema designs.

6.2.1 Analysis of Update and Join Efficiency Levels. We applied Algorithms 2 (Opt), 3 (Greed), and 4 (Hybrid) to the schemata for *ncvoter* and *china*.

Table 10 shows the levels $\ell^U; \ell^J$ for the decompositions $\mathcal{D}$ and the original input, their total levels $\ell^{U,\text{total}}; \ell^{J,\text{total}}$, output size $|\mathcal{D}|$, and the runtime.

The reduction in update inefficiency levels is significant for Opt and Hybrid. Greed achieves small reductions due to lost FDs. The join efficiency level for Greed equals 1 since every FD becomes either a key or is lost. Under FD-preservation, Opt always achieves the optimum level ℓ^U, which always equals ℓ^J as all FDs are preserved. Hybrid achieves even lower levels ℓ^U by loss of some FDs and lowering ℓ^J.

The reductions of ℓ^U are achieved by different output sizes: Greed requires few schemata, while Opt achieves the optimum by adding schemata to preserve all FDs, and Hybrid achieves further reductions by even more schemata. All algorithms are fast as the input is the unique atomic closure [31].

6.2.2 Physical Performance. We ran updates and join queries over our original datasets and the outputs of Algorithms 2 and 3 on input sets that included all FDs $X \rightarrow Y$ where ℓ_X met a given target level ℓ. Figures 4 and 6 show the results of Algorithm 2, and Figures 5 and 7 those of Algorithm 3. The x-axes always show the target levels ℓ, the number n of output tables and the number of input FDs. The y-axes in Figures 4 and 5 show the runtimes (in seconds) for a join query, and the runtimes (in seconds) for updates in Figures 6 and 7.

The join queries are:

— SELECT * FROM Hap over the original schema Hap, and
— SELECT * FROM R_1 NATURAL JOIN ... NATURAL JOIN R_n over $\{R_1, \ldots, R_n\}$.

Join times increase with lower target levels ℓ. For Opt, join performance becomes poor due to the many output tables required for FD preservation. This is compensated by Greed, but the tradeoff is poor update performance for lost FDs. The level ℓ, solely determined by the input integrity constraints, translates into different physical performance degrees for updates and joins on the output of our algorithms. Making these options available is the point of our new framework.

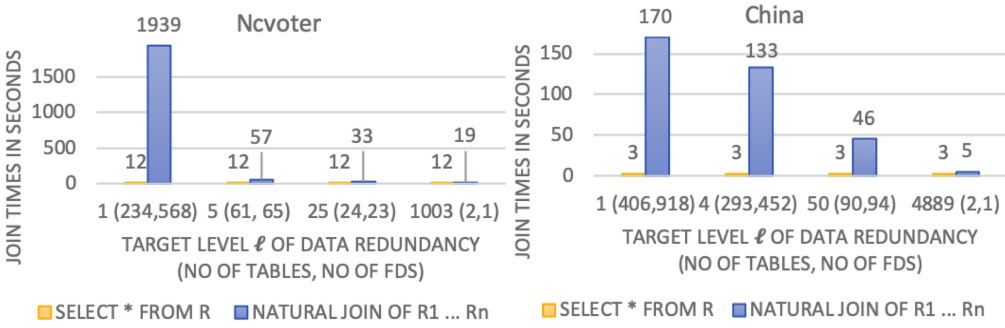


Fig. 4. Join performance after using Opt.

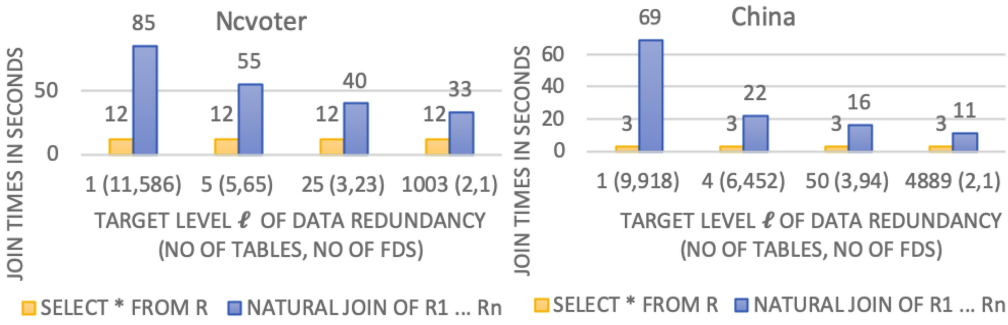


Fig. 5. Join performance after using Greed.

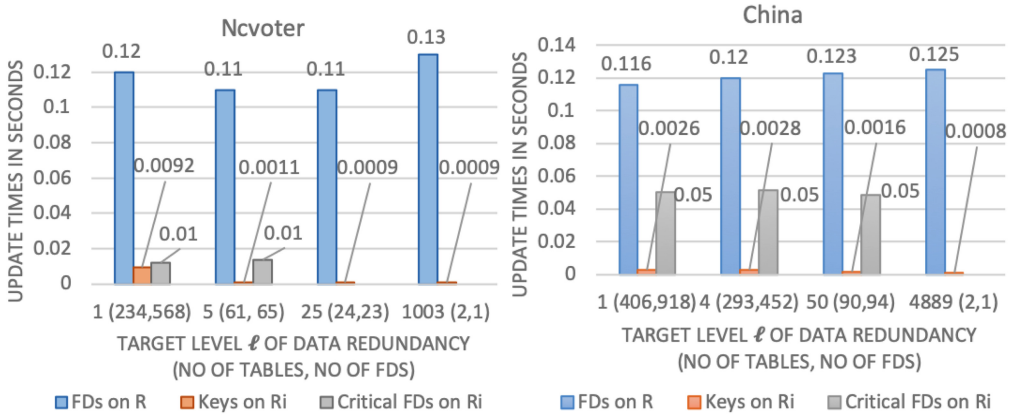


Fig. 6. Update performance after using Opt.

Updates involve changes to all occurrences of some redundant value (recall the update scenario from Section 4). After an FD is transformed into a key, every value occurs at most once, so an update is required for at most one occurrence.

Average update times for keys are about two orders of magnitude lower than those for input FDs on the original data. For preserved non-key FDs the gain is about one order of magnitude (grey bars in Figures 6 and 7, with no data point when all FDs become keys). Lost FDs require

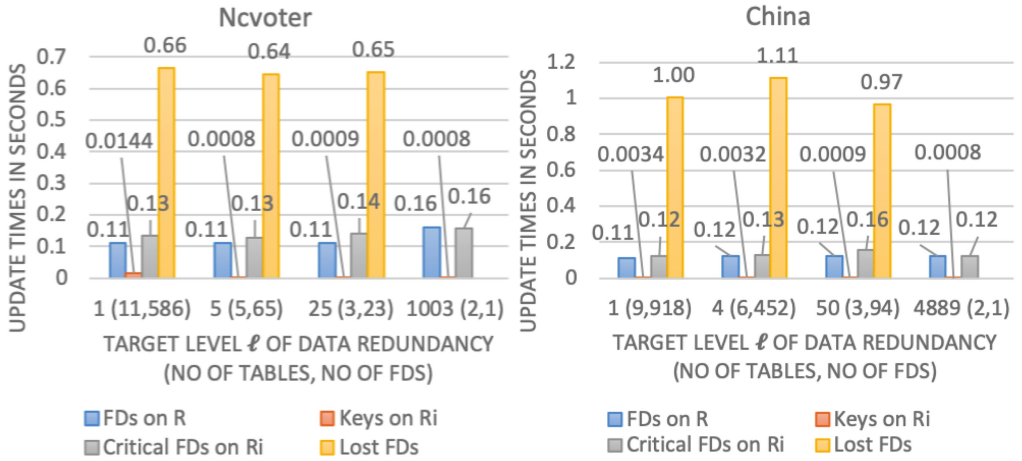


Fig. 7. Update performance after using Greed.

more update time than on the original dataset, due to necessary joins. With no lost FDs, only keys need to be enforced (in Figure 7 for the largest target level). Figures 6 and 7 show that, by lowering the level ℓ, more FDs are converted into keys (for Opt), which improves update efficiency at the expense of introducing more tables. A main message is that schema design is primarily driven by high-ranked FDs, but inclusion of low-ranked FDs means that more FDs need to be preserved during synthesis to lower their update time (by transformation into keys or non-key FDs with Opt), or more FDs will be lost during decomposition (Greed).

6.2.3 Comments. Logical schema design is driven by business rules which constrain instances to those that are meaningful to the underlying domain. We have seen how the upper bounds of CCs inform schema design and quantify the support for query and update operations. Our experiments illustrate that smaller bounds mean smaller differences in performance between designs. During logical design the choice of a schema is thus determined by the upper bounds, as these represent the current requirements. When requirements change over time, physical database tuning, such as de-normalization, can lift performance. However, this is only possible once the database is operational, when actual instances can be observed and stable patterns of workloads have emerged. If constraints evolve to the point that a different design is more suitable, then this may justify migration to a new schema. Ultimately, logical design informs the choice of a schema based on application requirements, independently of whether they have evolved or not.

Ranking designs informs the search for a relevant schema, just as ranking websites informs the search for relevant websites. This also extends to the definition and selection of materialized views. Our experiments suggest to carefully examine which operations are affected by which FDs. The level of data redundancy caused by the FDs quantifies how much they affect the operations. In particular, if the level of data redundancy caused by some FD is a priori unbounded ($\ell = \infty$), our framework alerts analysts to the fact that requirements analysis may still be incomplete.

6.3 Recommending Dimensions for Star Schema Designs

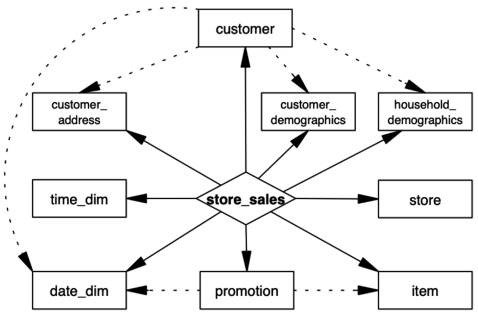
An important task in data profiling is the discovery of FDs that hold on a given dataset [1]. Research in this area is broad and deep [30, 53, 54, 67]. In particular, automated schema design has been identified as an important domain of application for data profiling [55]. In [67], the authors recommended ranking mined FDs by the number of redundant data values they cause, which is of

Table 11. Tables and Design of TPC-DS Benchmark

(a) Tables with store as the fact table

table	*#rows*	*#columns*
store_sales	2,880,403	23
customer	100,000	18
customer_address	50,000	13
customer_demographics	1,920,800	9
date_dim	73,049	28
household_demographics	7,200	5
item	18,000	22
promotion	300	19
time_dim	86400	10
store	12	29

(b) Star Schema Design



particular significance for recommending schema designs. As a proof of concept, we demonstrate how the level of data redundancy can inform the recommendation of dimension tables for star schema designs. Indeed, by mining FDs from the join of all star schema tables in TPC-DS, computing levels of data redundancy and numbers of redundant data value occurrences these FDs exhibit, and manually inspecting them, we can recover the original design of the star schema.

TPC-DS is a real-world like database benchmark designed for evaluating analytical queries. Table 11 shows the schemata in Table 11a and the star schema design in Table 11b. Straight edges reference dimension tables from the fact table while dotted edges show references between dimension tables. We joined instances of the schemata, and used exact FD discovery on the join that contains 2,880,403 rows and 167 columns. While we discovered more than 50,000 FDs, we excluded those where the LHS had more than seven attributes. Typically, such FDs are accidental and, even more important in our context, only exhibit very low levels of data redundancy. For example, the FD

ss_promo_sk, ss_ticket_number, ss_quantity, ss_wholesale_cost, ss_sales_price, ss_ext _wholesale_cost, ss_ext _list_price, ss_net_paid_inc_tax → ss_ext_discount_amt, ss_coupon_amt

causes only six redundant data value occurrences and the level of data redundancy is two.

Table 12 shows the non-key FDs whose attribute sets coincide with the dimension tables of Table 11a. We note that these exhibit significant maximum and average levels of redundancy, and large numbers of redundant data value occurrences. Within those dimension tables, there are other non-key FDs that also cause high levels of redundancy and numbers of redundant data value occurrences. Examples of such FDs are shown in Table 13. While these FDs may be used to create additional sub-dimensions leading to a snowflake design, this is not recommended here as we expect little to none updates of these redundant data values.

As a result of this semi-automated analysis, mining the join drives a recommendation for the original star schema design illustrated in Figure 11(b). It is in 1-Bounded Cardinality Normal Form, coinciding with BCNF.

7 Algebraic Characterization of Bounded Cardinality Normal Form

The aim of this section is to use lattice theory to establish a uniform algebraic characterization of both instances and schemata to be in ℓ-Bounded Cardinality Normal Form for any given ℓ. From a theoretical perspective, our characterizations open up the area for future mathematical inquiry, similar to how relational algebra has provided a foundation for query optimization. From a practical perspective, it is a never-ending task to align database instances and the business rules that govern them, which is important for data quality, analytics, and database performance. Following

Table 12. Non-key FDs *LHS* → *RHS* that Identify Dimension Tables and Their Properties

LHS	*RHS*	*dimension*	*max level*	*avg level*	*#redundancy*
p_promo_sk	p_promo_id, p_start_date_sk, p_end_date_sk, p_item_sk, p_cost, p_response_target, p_promo_name, p_channel_dmail, p_channel_email, p_channel_catalog, p_channel_tv, p_channel_radio, p_channel_press, p_channel_event, p_channel_demo, p_channel_details, p_purpose, p_discount_active	promotion	9,379	9,169	2,750,920
d_date_sk	d_date_id, d_date, d_month_seq, d_week_seq, d_quarter_seq, d_year, d_dow, d_moy, d_dom, d_qoy, d_fy_year, d_fy_quarter_seq, d_fy_week_seq, d_day_name, d_quarter_name, d_holiday, d_weekend, d_following_holiday, d_first_dom, d_last_dom, d_same_day_ly, d_same_day_lq, d_current_day, d_current_week, d_current_month, d_current_quarter, d_current_year	date_dim	4,074	1,508	2,750,311
i_item_sk	i_item_id, i_rec_start_date, i_rec_end_date, i_item_desc, i_current_price, i_wholesale_cost, i_brand_id, i_brand, i_class_id, i_class, i_category_id, i_category, i_manufact_id, i_manufact, i_size, i_formulation, i_color, i_units, i_container, i_manager_id, i_product_name	item	384	160	2,880,403
hd_demo_sk	hd_income_band_sk, hd_buy_potential, hd_dep_count, hd_vehicle_count	household_ demo graphics	647	382	2,750,556
t_time_sk	t_time_id, t_time, t_hour, t_minute, t_second, t_am_pm, t_shift, t_sub_shift, t_meal_time	time_dim	233	60	2,750,766
ca_address_sk	ca_address_id, ca_street_number, ca_street_name, ca_street_type, ca_suite_number, ca_city, ca_county, ca_state, ca_zip, ca_country, ca_gmt_offset, ca_location_type	customer_ address	185	55	2,750,429
c_customer_sk	c_customer_id, c_current_cdemo_sk, c_current_hdemo_sk, c_current_addr_sk, c_first_shipto_date_sk, c_first_sales_date_sk, c_salutation, c_first_name, c_last_name, c_preferred_cust_flag, c_birth_day, c_birth_month, c_birth_year, c_birth_country, c_login, c_email_address, c_last_review_date	customer	139	30	2,750,652
cd_demo_sk	cd_gender, cd_marital_status, cd_education_status, cd_purchase_estimate, cd_credit_rating, cd_dep_count, cd_dep_employed_count, cd_dep_college_count	customer_ demo graphics	63	12	2,750,704
s_store_sk	s_store_id, s_rec_start_date, s_rec_end_date, s_closed_date_sk, s_store_name, s_number_employees, s_floor_space, s_hours, s_manager, s_market_id, s_geography_class, s_market_desc, s_market_manager, s_division_id, s_division_name, s_company_id, s_company_name, s_street_number, s_street_name, s_street_type, s_suite_number, s_city, s_county, s_state, s_zip, s_country, s_gmt_offset, s_tax_precentage	store	459,491	458,394	2,750,369

a more detailed motivation in Section 7.1, we introduce the necessary concepts from lattice theory in Section 7.2. FDs and CCs are characterized algebraically in Section 7.3, before characterizing ℓ-BCNF on instances in Section 7.4 and on schemata in Section 7.5. Faithful translations from instances to schemata and vice versa are presented in Section 7.6. Our treatment and results subsume those from [39] for database instances, where keys occur as a special case of CCs, and BCNF as the special case of ℓ-Bounded Cardinality Normal Form where $\ell = 1$. However, we also introduce results for the schema level and the faithful translations not covered before.

Table 13. Non-key FDs *LHS* → *RHS* with Significant Levels of Data Redundancy Embedded in Dimension Tables

LHS	*RHS*	*max level*	*avg level*	*#redundancy*
d_year	d_fy_year	553,861	458,385	2,750,311
d_dow	d_day_name, d_weekend	394,704	392,901	2,750,311
d_day_name	d_weekend	394,704	392,901	2,750,311
t_hour	t_am_pm, t_shift, t_sub_shift, t_meal_time	331,501	211,597	2,750,766
d_quarter	seq_d_fy_quarter_seq	213,328	130,967	2,750,311

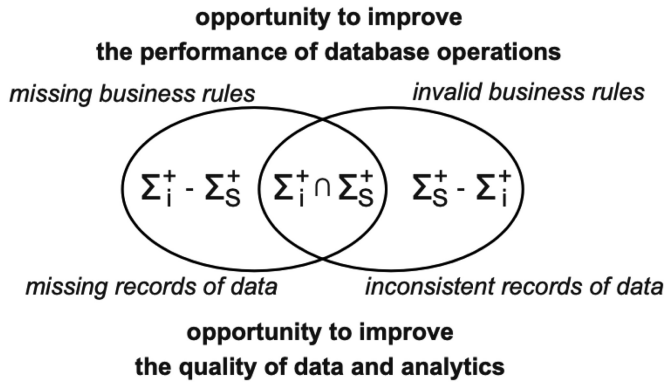


Fig. 8. Opportunities for optimization due to differences between database instances and business rules.

7.1 Motivation: Aligning Data and Schemata

Business rules are database constraints, such as CCs and FDs, that every database instance of an application domain ought to comply with. It is important for the quality of data, the rigor of insight derived from data analytics, and the performance of database operations that these business rules align with the database instances they govern. More formally, we may mine all database constraints (in a class of interest) that hold on a given database instance i, and denote their closure by Σ_i^+. Likewise, we may denote the closure of all database constraints (in the same class of interest) currently perceived as business rules on the corresponding database schema S by Σ_S^+. Ideally, Σ_i^+ and Σ_S^+ align perfectly, that is $\Sigma_i^+ = \Sigma_S^+$. In practice, however, this is rarely the case. This situation is depicted in Figure 8.

We may refer to $\Sigma_i^+ \cap \Sigma_S^+$ as the aligned core of business rules, keeping $\Sigma_i^+ = \Sigma_S^+$ as the perfect target. Indeed, constraints in $\Sigma_i^+ - \Sigma_S^+$ are satisfied by the database instance i, but are not currently perceived as business rules. Hence, it is the case for each of those constraints that they are either missing business rules, or that some records of data are missing (namely those that violate them). In the former case, there is an opportunity to declare new business rules while the latter case suggests opportunities to better data acquisition. Hence, we can improve the performance of database operations or the quality of data and analytics. Vice versa, constraints in $\Sigma_S^+ - \Sigma_i^+$ have been declared as currently perceived business rules but are not satisfied by the database instance i. Hence, it is the case for each of those constraints that they are either no longer business rules, perhaps due to the evolution of requirements, or there are records of data in i that are inconsistent with these rules. In the former case such rules must no longer be enforced to make the acquisition of new critical data possible and to stop their incorrect use for improving database performance, while the latter case leads naturally to the cleaning of data records. As before, we can better the performance of database operations or the quality of data and analytics.

With these opportunities in mind, we aim at characterizing ℓ-BCNF for both database instances and database schemata, and to translate one perspective into the other. Starting from the algebraic structures derived from instances, we obtain dual structures for the schemata derived from the set of CCs and FDs that hold on the instance. Vice versa, starting from the algebraic structures derived from schemata (including the given CCs and FDs), we obtain dual structures for the instances that satisfy the CCs and FDs of the given set and violate all those CCs and FDs not implied by the set.

7.2 Algebraic Foundations

We start by fixing basic notions and notation from algebra.

Let S denote a finite, non-empty set. A family $\pi = \{B_i \mid i \in I\}$ of subsets, called *blocks* of S, is said to form a *partition* of S if each B_i is non-empty; for all $i \neq j$ in I, $B_i \cap B_j = \emptyset$; and $\bigcup\{B_i \mid i \in I\} = S$.

A *partial order* $\leq$ of S is a binary relation $\leq \subseteq S \times S$ which is reflexive, anti-symmetric, and transitive. In this case, $(S, \leq)$ is a *partially ordered set*, or *poset*.

Given a poset $(S, \leq)$ and a subset U of S, we call $m \in S$ the *greatest lower bound (g.l.b.)* of U if and only if for all $u \in U$, $m \leq u$; and (for all $u \in U$, $m' \leq u$) implies that $m' \leq m$. Similarly, we call $m \in S$ the *least upper bound (l.u.b.)* of U if and only if for all $u \in U$, $u \leq m$; and (for all $u \in U$, $u \leq m'$) implies that $m \leq m'$.

A *lattice* is a poset in which any two elements a and b have a g.l.b., called a *meet* and denoted by $a \cdot b$, and an l.u.b., called a *join* and denoted by $a + b$. We sometimes write the meet $a \cdot b$ as ab if no confusion is created.

Let the set of all partitions π_i on S be denoted by $\Pi(S)$, and define a partial order on $\Pi(S)$ as follows: if (for all $a, b \in S$, $a\pi_1 b$ implies $a\pi_2 b$), then $\pi_1 \leq \pi_2$.

The poset $(\Pi(S), \leq)$ is seen to be a lattice $(\Pi(S), \leq, \cdot, +)$ with a universal lower bound $\mathbf{0} = \{B_i \mid i \in I\}$ such that every block B_i is a singleton, and a universal upper bound $\mathbf{1} = \{S\}$. When specifying a particular partition, we list the elements, and distinguish blocks with bars and semicolons. For example, if $S = \{1, 2, 3, 4, 5\}$ and partition π on S has blocks $\{1, 3, 4\}$ and $\{2, 5\}$, then we write $\pi = \{\overline{1, 3, 4}; \overline{2, 5}\}$. The meet and join of any two partitions $\pi_1, \pi_2 \in \Pi(S)$ can be determined as follows:

(1) For all $a, b \in S$, $a\pi_1 \cdot \pi_2 b$ iff $a\pi_1 b$ and $a\pi_2 b$.
(2) For all $a, b \in S$, $a\pi_1 + \pi_2 b$ iff there is some positive integer n and $c_0, \ldots, c_n \in S$ such that $a = c_0$, $b = c_n$ and $c_i \pi_1 c_{i+1}$ or $c_i \pi_2 c_{i+1}$ for each i, $0 \leq i \leq n - 1$.

A *Boolean algebra* is a lattice $(L, \leq, \cdot, +, \text{-}, \mathbf{0}, \mathbf{1})$ with the following properties:

— (distributive identities) for all $a, b, c \in L$, $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$, and $a + (b \cdot c) = (a + b) \cdot (a + c)$ hold.
— (modular identity) for all $a, b, c \in L$, $a \leq c$ implies $a + (b \cdot c) = (a + b) \cdot c$.
— (universal bounds) for all $a \in L$, $\mathbf{0} \cdot a = \mathbf{0}$, $\mathbf{0} + a = a$, $\mathbf{1} \cdot a = a$, and $\mathbf{1} + a = 1$.
— (complements) for all $a \in L$, there is some $\overline{a} \in L$ such that $a \cdot \overline{a} = \mathbf{0}$, $a + \overline{a} = \mathbf{1}$, and $\overline{\overline{a}} = a$, and for all $b \in L$, $\overline{a \cdot b} = \overline{a} + \overline{b}$ and $\overline{a + b} = \overline{a} \cdot \overline{b}$.

The set of all subsets of S, called the *power set* of S, and denoted by 2^S, with the partial order $\forall S_1, S_2 \in 2^S$ ($S_1 \leq S_2$ iff $S_2 \subseteq S_1$), is a Boolean algebra $(2^S, \leq, \cdot, +, \text{-}, \mathbf{0}, \mathbf{1})$ with the universal bound $\mathbf{0} = S$ and $\mathbf{1} = \emptyset$. The *dual* of a poset is the poset with the converse partial order on the same elements. The Boolean algebra defined above is the dual of the conventional Boolean algebra of the power set. The operations of meet and join are defined by

(1) Meet (g.l.b.) $S_1 \cdot S_2 = S_1 \cup S_2$
(2) Join (l.u.b.) $S_1 + S_2 = S_1 \cap S_2$,

and the complement of $S_1 \in 2^S$ is $\overline{S_1} = S - S_1$.

Table 14. Relation r over Book={*ISBN*,*Title*,*Genre*,*Supplier*} with Identifier for Each Tuple in r

Tuple_ID	*ISBN*	*Title*	*Genre*	*Supplier*
1	0143035002	Anna Karenina	Classic	Penguin
2	0143035002	Anna Karenina	Classic	Alibaba
3	0486821951	Don Quixote	Fiction	Amazon
4	0060934344	Don Quixote	Fiction	Amazon
5	1607107333	Don Quixote	Fiction	Amazon
6	2266082701	Le Rouge Et le Noir	Novel	Presses Pocket
7	9782266082709	Le Rouge Et le Noir	Literature	Presses Pocket
8	1250849861	Friends Forever	Children	Google
9	1250753961	Best Friends	Children	Google

Let $\psi : L \to M$ be a function from lattice L to lattice M. We say ψ is a *meet-morphism* if $\forall a, b \in L, \psi(a \cdot b) = \psi(a) \cdot \psi(b)$, and ψ is a *join-morphism* if $\forall a, b \in L, \psi(a + b) = \psi(a) + \psi(b)$. Meet-morphisms and join-morphisms are both *isotone* (order-preserving); that is, $\forall a, b \in L, a \leq b$ implies $\psi(a) \leq \psi(b)$, and any order-preserving one-to-one mapping with an inverse is an *isomorphism*.

Let r be a relation on the set R of attributes. The set of all subsets of R, denoted by 2^R, with the partial order defined by set-containment is a Boolean algebra $(2^R, \supseteq, \cdot, +, -, S, \emptyset)$, where the meet, join, and complement operations are defined as above. For every $X \in 2^R$, there is a partition on the set of tuples in r defined as follows [39].

Definition 7.1. Let r be a relation on the set R of attributes. Each subset of R is associated with a partition of r. We define the function $\pi : 2^R \to \Pi(r)$, which we call the *partition function* associated with r, by $\pi : X \mapsto \pi_X = r/\theta(X)$ where $\theta(X) = \{(t_1, t_2) \in r \times r \mid t_1[X] = t_2[X]\}$. □

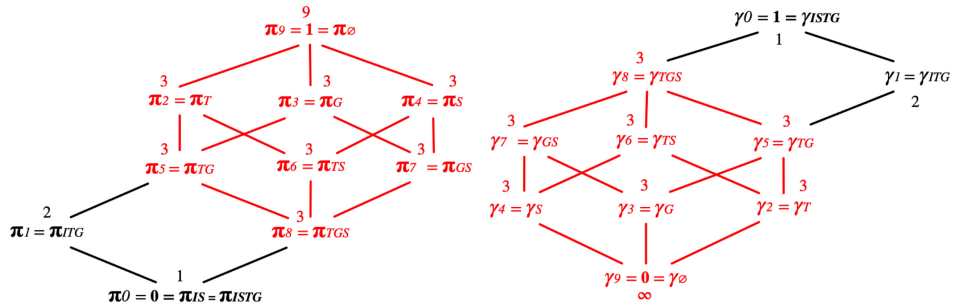
In general, the image set $Im(\pi)$ of π is not a sublattice of $\Pi(r)$. Since $\pi_1 \cdot \pi_2 \in Im(\theta)$, $Im(\theta)$ is a complete lattice, and we call it the *relation lattice* of r, denoted by L_r. Since r does not contain any duplicate tuples, $\pi_R = r/\theta(R) = \mathbf{0}$, and since tuples cannot be distinguished by the empty set of attributes, we define $\pi_\emptyset = \theta(\emptyset) = \mathbf{1}$. Hence, the universal bounds of L_r are the same as those in $\Pi(r)$. It is known that the partition function $\pi : 2^R \to L_r$ is a meet-morphism, and order-preserving. However, it is not a join-morphism, in general. The relation $\pi \circ \pi^{-1}$ on 2^R defined by

$$\pi \circ \pi^{-1} = \{(X, Y) \in 2^R \times 2^R \mid \pi_X = \pi_Y\}$$

is an equivalence relation, and sets in the quotient set $2^R/\pi \circ \pi^{-1}$ will be called π *classes*.

Example 7.2. We illustrate the concepts on a new example over schema Book that manages basic information about books, in the form of their *I(SBN)*, *T(itle)*, *G(enre)*, and *S(upplier)*. Table 14 shows a relation r that will serve us as an instance of the schema on which we illustrate our concepts. The partition lattice L_r of r consists of the following partitions:

— $\pi_0 = \mathbf{0} = \pi_{IS} = \pi_{ITS} = \pi_{IGS} = \pi_{ITGS} = \{\overline{1}; \overline{2}; \overline{3}; \overline{4}; \overline{5}; \overline{6}; \overline{7}; \overline{8}; \overline{9}\}$
— $\pi_1 = \pi_I = \pi_{IT} = \pi_{IG} = \pi_{ITG} = \{\overline{1, 2}; \overline{3}; \overline{4}; \overline{5}; \overline{6}; \overline{7}; \overline{8}; \overline{9}\}$
— $\pi_2 = \pi_T = \{\overline{1, 2}; \overline{3, 4, 5}; \overline{6, 7}; \overline{8}; \overline{9}\}$
— $\pi_3 = \pi_G = \{\overline{1, 2}; \overline{3, 4, 5}; \overline{6}; \overline{7}; \overline{8, 9}\}$
— $\pi_4 = \pi_S = \{\overline{1, 2}; \overline{3, 4, 5}; \overline{6, 7}; \overline{8, 9}\}$
— $\pi_5 = \pi_{TG} = \{\overline{1, 2}; \overline{3, 4, 5}; \overline{6}; \overline{7}; \overline{8}; \overline{9}\}$
— $\pi_6 = \pi_{TS} = \{\overline{1, 2}; \overline{3, 4, 5}; \overline{6, 7}; \overline{8}; \overline{9}\}$
— $\pi_7 = \pi_{GS} = \{\overline{1, 2}; \overline{3, 4, 5}; \overline{6}; \overline{7}; \overline{8, 9}\}$
— $\pi_8 = \pi_{TGS} = \{\overline{1, 2}; \overline{3, 4, 5}; \overline{6}; \overline{7}; \overline{8}; \overline{9}\}$
— $\pi_9 = \mathbf{1} = \pi_\emptyset = \{\overline{1, 2, 3, 4, 5, 6, 7, 8, 9}\}$



(a) Partition lattice with levels of partitions (b) Closure lattice with levels of closed sets

Fig. 9. Lattices associated with relations and schemata in ℓ-BCNF.

The Hasse diagram of the partition lattice is illustrated in Figure 9(a). For now, only the vertices and edges are important. The numbers attached to vertices and the reason for coloring parts of the lattice in red will be discussed subsequently. □

7.3 Functional Dependencies and Cardinality Constraints

The algebraic framework provides us directly with a different point of view for the semantics of FDs and CCs. We will give definitions from this point of view, and then validate the inference rules of our axiomatization.

The algebraic formulation of the semantics for FDs is already known [39].

Lemma 7.3. *Let π denote the partition function associated with the relation r over R, and let $X \to Y$ denote an FD over R. Then r satisfies $X \to Y$ if and only if $\pi_X \leq \pi_Y$.* □

Example 7.4. Given the relation r from Table 14, one may verify that r satisfies the FDs *ISBN* $\to$ *Title* and *ISBN* $\to$ *Genre*, but does satisfy neither the FD *Title, Genre, Supplier* $\to$ *ISBN* nor the FD *Genre, Supplier* $\to$ *Title*. In fact, the Hasse diagram in Figure 9(a) shows us directly that both $\pi_I \leq \pi_T$ and $\pi_I \leq \pi_G$ hold, but neither $\pi_{TGS} \leq \pi_I$ nor $\pi_{GS} \leq \pi_T$ hold.

The semantics of CCs has not been studied yet from an algebraic point of view. Before we define an algebraic semantics, we introduce a new concept into the algebraic framework.

Definition 7.5. Let π denote the partition function associated with the relation r over R, and let $X \in 2^R$. We define *the level* ℓ_{π_X} *of* π *modulo* X as the maximum block size in the partition π_X of L_r, that is, $\ell_{\pi_X} := \max\{|B| \mid B \in \pi_X\}$. □

Note that $\ell_{\pi_\emptyset} = \ell_{\mathbf{1}} = |r|$ and $\ell_{\pi_R} = \ell_{\mathbf{0}} = 1$ appear as special cases of Definition 7.5.

Example 7.6. The following denote all levels of partitions in L_r for the relation r from Table 14: $\ell_{\pi_\emptyset} = 1$, $\ell_{\pi_1} = 2$, $\ell_{\pi_9} = 9$, and $\ell_{\pi_i} = 3$ for $i = 2, \ldots, 8$. These levels are also part of Figure 9(a) where they are attached to each of the vertices, respectively.

We may now state the semantics of CCs in our algebraic framework.

Lemma 7.7. *Let π denote the partition function associated with the relation r over relation schema R, and let $card(X) \leq \ell$ denote a CC over R. Then r satisfies $card(X) \leq \ell$ if and only if $\ell_{\pi_X} \leq \ell$.*

The semantics of keys and minimal keys has been analyzed from an algebraic point of view [39]. A key X is covered as the special case of a CCs $card(X) \leq \ell$ if and only if $\ell = 1$. Indeed, $\ell = 1$ if and only if $\ell_{\pi_X} = 1$ if and only if $\pi_X = \mathbf{0}$.

Example 7.8. Given the relation r from Table 14, one may verify that r satisfies the CCs $card(IS) \leq 1$ and $card(TGS) \leq 3$, but does satisfy neither the CC $card(ITG) \leq 1$ nor the CC $card(TGS) \leq 2$. In fact, the Hasse diagram in Figure 9(a) shows us directly that both $\ell_{\pi_{IS}} = 1 \leq 1$ and $\ell_{\pi_{TGS}} = 3 \leq 3$ hold, but neither $\ell_{\pi_{ITG}} = 2 \leq 1$ nor $\ell_{\pi_{TGS}} = 3 \leq 2$ hold.

As an interesting exercise, we may use the algebraic framework to verify the soundness of inference rules in our axiomatization $\mathfrak{Q}$ from Table 4. Note that the Armstrong axioms from Table 3 have already been validated from the algebraic perspective [39].

The *set axiom* $\mathcal{S}$, $card(R) \leq 1$, is sound since $\ell_{\pi_R} = 1 \leq 1$. Indeed, no duplicate tuples can occur in any relation, that is, there cannot be different tuples in a relation with matching values on all attributes.

For the *unbounded* axiom $\mathcal{U}$, we have $\ell_{\pi_X} = \ell$ for some finite positive integer ℓ since r is finite. Hence, $\ell_{\pi_X} \leq \infty$.

For the *loosen* inference rule $\mathcal{L}$, we assume that r satisfies $card(X) \leq \ell$. Consequently, $\ell_{\pi_X} \leq \ell$, and therefore $\ell_{\pi_X} \leq \ell + 1$, which means that r satisfies also $card(X) \leq \ell + 1$.

For the *adding* inference rule $\mathcal{A}$, we assume that r satisfies $card(X) \leq \ell$. Consequently, $\ell_{\pi_X} \leq \ell$, and therefore also $\ell_{\pi_{XY}} \leq \ell_{\pi_X} \leq \ell$, which means that r satisfies also $card(XY) \leq \ell$.

For the *key* inference rule $\mathcal{K}$, we assume that r satisfies $card(X) \leq 1$. Consequently, $\ell_{\pi_X} = 1$, and therefore $\pi_X = \mathbf{0} \leq \pi_Y$ for every $Y \subseteq R$. Hence, r satisfies also $X \to Y$.

Finally, for the *pullback* inference rule $\mathcal{P}$, we assume that r satisfies $X \to Y$ and $card(Y) \leq \ell$. Consequently, $\ell_{\pi_X} \leq \ell_{\pi_Y}$ by Lemma 7.3 and $\ell_{\pi_Y} \leq \ell$ by Lemma 7.7, respectively. Hence, $\ell_{\pi_X} \leq \ell$, which means $\models_r card(X) \leq \ell$.

We will now apply the algebraic framework to our ℓ-Bounded Cardinality Normal Form, starting with the instance level.

7.4 Characterization of ℓ-BCNF for Instances

We first define when a relation is in ℓ-Bounded Cardinality Normal Form.

Definition 7.9. Let r be a relation over relation schema R. We say that r *is in ℓ-Bounded Cardinality Normal Form* if and only if for all FDs $X \to Y$ satisfied by r, $\ell_{\pi_X} \leq \ell$ or $Y \subseteq X$. □

Example 7.10. In Table 14, the relation r is not in ℓ-BCNF for $\ell = 1$. Indeed, r satisfies the non-trivial FD *ISBN* $\to$ *Title*, but $\ell_{\pi_I} = 2$. One may inspect all other non-trivial FDs $X \to Y$ that are satisfied by r, which are implied by *ISBN* $\to$ *Title* and *ISBN* $\to$ *Genre*. For each of these FDs it follows that $\ell_{\pi_X} = 2$. Consequently, r is in 2-BCNF.

We want to show that a relation lattice has some special properties when the underlying relation is in ℓ-Bounded Cardinality Normal Form. This requires us to introduce some more useful terminology from algebra. A subset $F \subseteq L$ for a lattice $(L, \leq, \cdot, +)$ is a *filter* if

— For all $a \in F$, $b \in L$, $b \geq a$ implies $b \in F$.
— For all $a, b \in F$, $a \cdot b \in F$.

For every $a \in L$, the subset of all elements "greater than or equal to" a is a filter; called the *principal filter* of L generated by a, and is denoted by $[a)$, that is, $[a) = \{b \in L \mid b \geq a\}$.

Definition 7.11. Let r denote a relation over relation schema R, and let π denote a partition of the relation lattice L_r. Let $R_\pi = \{A \in R \mid \pi \leq \pi_A\} \subseteq R$. Then the projection $r[R_\pi]$ of R onto R_π is called a *projection* and $[\pi)$ is called a *filter of level* ℓ_π.

Example 7.12. Given the partition lattice in Figure 9(a), a principal filter $[\pi)$ generated by π is obtained by including π in $[\pi)$ and closing it upwards. For example $[\pi_8) = \{\pi_8, \pi_5, \pi_6, \pi_7, \pi_2, \pi_3, \pi_4, \pi_9\}$, which is marked in red in the figure. The level of $[\pi_8)$ is $\ell_{[\pi_8)} = 3$.

The principal filter of the last example appears to enjoy particularly nice properties that distinguish it from other filters such as $[\pi_1)$. For example, the only FDs that hold on the projection are trivial. Note that this is not the case for $[\pi_1)$ since the non-trivial FD $I \rightarrow G$ holds on $r[R_{\pi_1}] = r[ITG]$. Hence, we record the following definition. While [39] required normality only for principal filters generated by atoms, we require the definition for all principal filters and can directly use our notion of levels for that.

Definition 7.13. Let r denote a relation over relation schema R, and let π denote a partition of the relation lattice L_r with level $\ell_\pi > 1$. The principal filter $[\pi)$ of L_r is called *normal* if and only if whenever the FD $X \rightarrow Y$ holds on the projection $r[R_\pi]$, then $Y \subseteq X$; otherwise, it is called *abnormal*. □

We can now record our first special algebraic property for instances in ℓ-Bounded Cardinality Normal Form. In the special case where $\ell = 1$, it is sufficient to consider all atomic filters instead of all principal filters from level 2 onwards [39].

Lemma 7.14. *A relation r over R is in ℓ-Bounded Cardinality Normal Form if and only if every principal filter $[\pi)$ of L_r with level $\ell_\pi > \ell$ is normal.*

Example 7.15. Given the partition lattice L_r of r in Figure 9(a), the only abnormal principal filter of level larger than 1 is π_1. This means that r is ℓ-BCNF if and only if $\ell > 1$.

Our next algebraic property shows that relations in ℓ-BCNF enjoy join-morphisms on certain elements of their partition lattice, as identified by ℓ.

Corollary 7.16. *If r is in ℓ-BCNF, $\ell_{\pi_X} > \ell$ and $\ell_{\pi_Y} > \ell$, then $\pi_{X+Y} = \pi_X + \pi_Y$.*

We illustrate the last corollary on the book case of Example 7.4.

Example 7.17. Consider the partition lattice from Figure 9(a) for the relation r from Table 14. As r is in 2-BCNF, we will get a join-morphism for any partitions above level 2. For example, taking $\pi_6 = \pi_{TS}$ and $\pi_7 = \pi_{GS}$ of level 3, we have $\pi_{TS+GS} = \pi_S$ which is indeed equal to $\pi_{TS} + \pi_{GS} = \pi_S$.

The first main result of this section is the following algebraic characterization of ℓ-Bounded Cardinality Normal Form. For the special case where $\ell = 1$, the result was shown in [39].

Theorem 7.18. *A relation r over R is in ℓ-BCNF if and only if every principal filter $[\pi)$ of level $\ell_\pi > \ell$ is isomorphic to the Boolean algebra $(2^{R_\pi}, \leq, \cdot, +, -, R_\pi, \mathbf{1})$.*

Example 7.19. The relation r of Table 14 is indeed in 2-BCNF. In fact, one may verify that each principal filter $[\pi)$ of level larger than 2 is isomorphic to the (dual of the) finite power set Boolean algebra with $|R_\pi|$-many elements, as illustrated in Figure 9(a) for the (dual of the) power set Boolean algebra $(2^{TGS}, \supseteq, \cup, \cap, -, TGS, \emptyset)$ generated by the principal filter $[\pi_8)$. It is also observable easily, that the principal filter $[\pi_1)$ of level 2 is not isomorphic to a Boolean algebra. Indeed, r is not in 1-BCNF.

Galois Connection. For a given relation r, the partition function $\pi : 2^R \rightarrow L_r$ maps $X \in 2^R$ to the partition π_X. Vice versa, $\rho : L_r \rightarrow 2^R$ maps $\pi \in L_r$ to $\bigcup\{Y \in 2^R \mid \pi \leq \pi_Y\} \in 2^R$. For all $\pi_X \in L_r$ and all $Y \in 2^R$ it is not difficult to see that $\rho(\pi_X) \leq Y$ iff $\pi_X \leq \pi_Y$ holds, since both sides are equivalent to r satisfying the FD $X \rightarrow Y$. It follows that ρ and π define a *monotone Galois connection* between the posets $(L_r, \leq)$ and $(2^R, \supseteq)$. As a consequence, the composition $\rho \circ \pi : 2^R \rightarrow 2^R$ defines a *closure operator*.

7.5 Characterization of ℓ-BCNF for Schemata

The previous sections have analyzed the algebraic properties associated with instances that are in ℓ-Bounded Cardinality Normal Form. This adds an interesting view that complements the traditional primary focus on the schema level. The latter is what we will analyze algebraically in this section.

For this purpose, we assume a fixed relation schema R together with a set Σ of CCs and FDs over R. The partition lattice served the purpose of partitioning the given relation by identifying tuples with matching values on a given set of attributes. Similarly, we will now utilize the closure lattice serving the purpose of partitioning the family of attribute subsets into those with the same closure, based on the given set Σ.

Definition 7.20. Let Σ be a set of CCs and FDs over the relation schema R. The function $\gamma : 2^R \to 2^R$, known as the *closure function* of Σ, is defined by $\gamma_X := \gamma(X) = \{A \in R \mid \Sigma \models X \to A\}$. An element $X \in 2^R$ is called *closed* when $\gamma_X = X$. The set $L_\Sigma := \{X \in 2^R \mid \gamma_X = X\}$ is the set of closed subsets of R. The complete lattice $(L_\Sigma, \subseteq, \cap, \cup, \gamma_\emptyset, R)$ is called the *closure lattice* of R for Σ. □

The closure lattice of Definition 7.20 is indeed well-defined. That is, γ is a closure operator on the poset $(2^R, \subseteq)$, satisfying for all $X, Y \in 2^R$ the axioms of (monotonicity) $X \subseteq \gamma(X)$, (idempotency) $\gamma(\gamma(X)) = \gamma(X)$, and (order-preservation) $X \subseteq Y$ means that $\gamma(X) \subseteq \gamma(Y)$. Hence, since the powerset Boolean algebra $(2^R, \subseteq, \cap_R, \cup_R, \emptyset, R)$ is a complete lattice, so is $(L_\Sigma, \subseteq, \cap, \cup, \gamma_\emptyset, R)$ where $X \cap Y = X \cap_R Y$ and $X \cup Y = \gamma(X \cup_R Y)$. In fact, the closure function $\gamma : 2^R \to L_\Sigma$ is a meet-morphism, and order-preserving. However, it is not a join-morphism, in general. The function $\gamma \circ \gamma^{-1}$ on 2^R defined by

$$\gamma \circ \gamma^{-1} = \{(X, Y) \in 2^R \times 2^R \mid \gamma_X = \gamma_Y\}$$

is an equivalence relation, and sets in the quotient set $2^R / \gamma \circ \gamma^{-1}$ will be called γ *classes*.

Example 7.21. We illustrate the concepts on our previous schema Book that manages basic information about books, in the form of their *I(SBN)*, *T(itle)*, *G(enre)*, and *S(upplier)*. Let Σ denote the set with the following CCs and FDs:

$$ISBN \to Title, ISBN \to Genre, card(Supplier) \leq 3, card(Title) \leq 3, card(Genre) \leq 3, card(ISBN) \leq 2.$$

The closure lattice L_Σ of Book consists of the following closed subsets:

$$\gamma_0 = \gamma_{ISTG} = ISTG, \gamma_1 = \gamma_{ITG} = ITG, \gamma_2 = \gamma_T = T, \gamma_3 = \gamma_G = G, \gamma_4 = \gamma_S = S, \gamma_5 = \gamma_{TG} = TG,$$
$$\gamma_6 = \gamma_{TS} = TS, \gamma_7 = \gamma_{GS} = GS, \gamma_8 = \gamma_{TGS} = TGS, \gamma_9 = \gamma_\emptyset = \emptyset.$$

The Hasse diagram of the closure lattice is illustrated in Figure 9(b). For now, only the vertices and edges are important. The numbers attached to vertices and the reason for coloring parts of the lattice in red will be discussed subsequently. □

The algebraic perspective of closed sets leads to a familiar characterization for the implication of FDs, namely that $\Sigma \models X \to Y$ if and only if Y is a subset of the attribute set closure of X [3].

Lemma 7.22. *Let γ denote the closure function associated with the set Σ of CCs and FDs over R, and let $X \to Y$ denote an FD over R. Then $\Sigma \models X \to Y$ if and only if $Y \subseteq \gamma_X$.* □

Example 7.23. Given the set Σ from Example 7.21, one may verify that Σ implies the FDs *ISBN* $\to$ *Title* and *ISBN* $\to$ *Genre*, but does imply neither the FD *Title, Genre, Supplier* $\to$ *ISBN* nor the FD *Genre, Supplier* $\to$ *Title*. In fact, the Hasse diagram in Figure 9(b) shows us directly that both $T \subseteq \gamma_I = ITG$ and $G \subseteq \gamma_I = ITG$ hold, but neither $I \subseteq \gamma_{TGS} = TGS$ nor $T \subseteq \gamma_{GS} = GS$ hold.

The semantics of CCs has not been studied from an algebraic point of view. Before we can establish a definition, we introduce a new concept into the algebraic framework.

Definition 7.24. Let γ denote the closure function associated with the set Σ of CCs and FDs over R, and let $X \in 2^R$. We define *the level* ℓ_{γ_X} *of* γ *modulo* X as the minimum lower bound ℓ that applies to γ_X implied by Σ, that is, $\ell_{\gamma_X} := \min\{\ell \mid \Sigma \models card(\gamma_X) \leq \ell\}$. □

Note that $\ell_{\gamma_R} = 1$ appears as a special case of Definition 7.24. In addition, if there is no finite integer ℓ such that $\Sigma \models card(\gamma_X) \leq \ell$, then $\ell_{\gamma_X} := \infty$.

Example 7.25. The following denote all levels of closed sets in L_Σ for the set Σ of CCs and FDs from Example 7.21: $\ell_{\gamma_0} = 1$, $\ell_{\gamma_1} = 2$, $\ell_{\gamma_9} = \infty$, and $\ell_{\gamma_i} = 3$ for $i = 2, \ldots, 8$. These levels are also part of Figure 9(b) where they are attached to each of the vertices, respectively.

We may now state the semantics of CCs in our algebraic framework.

Lemma 7.26. *Let* γ *denote the closure function associated with the set* Σ *of CCs and FDs over relation schema* R*, and let* $card(X) \leq \ell$ *denote a CC over* R*. Then* Σ *implies* $card(X) \leq \ell$ *if and only if* $\ell_{\gamma_X} \leq \ell$.

A key X is covered as the special case of a CC $card(X) \leq \ell$ iff $\ell = 1$. Indeed, $\ell = 1$ if and only if $\ell_{\gamma_X} = 1$.

Example 7.27. Given the set Σ from Example 7.21, one may verify that Σ implies the CCs $card(IS) \leq 1$ and $card(TGS) \leq 3$, but does imply neither the CC $card(ITG) \leq 1$ nor the CC $card(TGS) \leq 2$. In fact, the Hasse diagram in Figure 9(b) shows us directly that both $\ell_{\gamma_{IS} = \gamma_{ISTG}} = 1 \leq 1$ and $\ell_{\gamma_{TGS}} = 3 \leq 3$ hold, but neither $\ell_{\gamma_{ITG}} = 2 \leq 1$ nor $\ell_{\pi_{TGS}} = 3 \leq 2$ hold.

We now turn to an algebraic characterization of ℓ-Bounded Cardinality Normal Form on the schema level.

Proposition 7.28. (R, Σ) *is in* ℓ*-BCNF iff for all FDs* $X \to Y$ *implied by* Σ*,* $\ell_{\gamma_X} \leq \ell$ *or* $Y \subseteq X$.

Example 7.29. In Example 7.21, the set Σ of CCs and FDs is not in ℓ-BCNF for $\ell = 1$. Indeed, Σ contains the non-trivial FD *ISBN* $\to$ *Title*, but $\ell_{\gamma_I} = 2$. In addition, for the non-trivial FD *ISBN* $\to$ *Genre* we have $\ell_{\gamma_I} = 2$ as well. Consequently, R is in 2-BCNF for Σ.

We want to show that a closure lattice has some special properties when the underlying schema is in ℓ-Bounded Cardinality Normal Form. This requires us to introduce some more useful terminology from algebra. A subset $I \subseteq L$ for a lattice $(L, \leq, \cdot, +)$ is an *ideal* if

— For all $a \in I$, $b \in L$, $b \leq a$ implies $b \in I$.
— For all $a, b \in I$, $a + b \in I$.

For every $a \in L$, the subset of all elements "less than or equal to" a is an ideal; called the *principal ideal* of L generated by a, and is denoted by $(a]$, that is, $(a] = \{b \in L \mid b \leq a\}$.

Definition 7.30. Let Σ denote a set of CCs and FDs over relation schema R, and let γ_X denote a closed set of L_Σ. Let $R_{\gamma_X} = \{A \in R \mid \gamma_A \leq \gamma_X\} \subseteq R$. Then the projection $\Sigma[R_{\gamma_X}]$ of Σ onto R_{γ_X} is called an *FD projection* and $(\gamma_X]$ is called an *ideal of level* ℓ_{γ_X}.

Example 7.31. Given the closure lattice in Figure 9(b), a principal ideal $(\gamma_X]$ generated by γ_X is obtained by including γ_X in $(\gamma_X]$ and closing it downwards. For example $(\gamma_8] = \{\gamma_8, \gamma_7, \gamma_6, \gamma_5, \gamma_4, \gamma_3, \gamma_2, \gamma_9\}$, which is marked in red in the figure. The level of $(\gamma_8]$ is $\ell_{(\gamma_8]} = 3$.

The principal ideal of the last example enjoys properties that distinguish it from other ideals such as $(\gamma_1]$. For example, all FDs satisfied on the projection are trivial, which is not the case for $(\gamma_1]$ since the non-trivial FD $I \to G$ holds on $\Sigma[R_{\gamma_1}] = \Sigma[ITG]$. Hence, we record the following definition.

Definition 7.32. Let Σ denote a set of CCs and FDs over relation schema R, and let γ_Z denote an element of L_Σ with level $\ell_{\gamma_Z} > 1$. The principal ideal $(\gamma_Z]$ of L_r is called *normal* iff whenever the FD $X \to Y$ holds on the FD projection $\Sigma[R_{\gamma_Z}]$, then $Y \subseteq X$; otherwise, it is called *abnormal*. □

In analogy to Lemma 7.14 on the instance level, we now record our first special algebraic property for schemata in ℓ-Bounded Cardinality Normal Form.

Lemma 7.33. *(R, Σ) is in ℓ-Bounded Cardinality Normal Form if and only if every principal ideal $(\gamma_X]$ of L_Σ with level $\ell_{\gamma_X} > \ell$ is normal.*

Example 7.34. Given the closure lattice L_Σ of Σ in Figure 9(b), the only abnormal principal ideal of level larger than 1 is γ_1. This means that (R, Σ) is in ℓ-BCNF if and only if $\ell > 1$.

In analogy to Corollary 7.16 for the instance level, our next algebraic property shows that schemata in ℓ-BCNF enjoy join-morphisms on certain elements of their closure lattice, as identified by ℓ.

Corollary 7.35. *If (R, Σ) is in ℓ-BCNF, $\ell_{\gamma_X} > \ell$ and $\ell_{\gamma_Y} > \ell$, then $\gamma_{X \cup Y} = \gamma_X \cup \gamma_Y$.*

We illustrate the last corollary on the book case of Example 7.21.

Example 7.36. Consider the closure lattice from Figure 9(b) for the set Σ of CCs and FDs from Example 7.21. As (R, Σ) is in 2-BCNF, we will get a join-morphism for any closed sets above level 2. For example, taking $\gamma_6 = \gamma_{TS}$ and $\gamma_7 = \gamma_{GS}$ of level 3, we have $\gamma_{TS \cup GS} = \gamma_{TGS}$ which is indeed equal to $\gamma_{TS} \cup \gamma_{GS} = \gamma_{TGS}$.

Following Theorem 7.18 on the instance level, the second main result of this section is the following algebraic characteristic of ℓ-Bounded Cardinality Normal Form at the schema level.

Theorem 7.37. *(R, Σ) is in ℓ-BCNF if and only if every principal ideal $(\gamma_Z]$ of level $\ell_{\gamma_Z} > \ell$ is isomorphic to the Boolean algebra $(2^{R_{\gamma_Z}}, \subseteq, \cap, \cup, -, \emptyset, R_{\gamma_Z})$.*

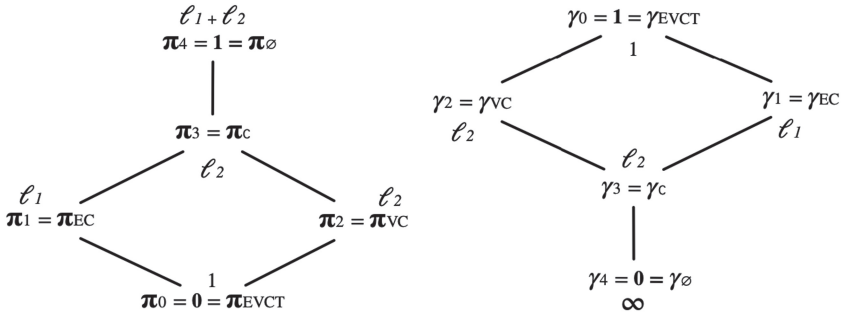
Example 7.38. (R, Σ) of Example 7.21 is indeed in 2-BCNF. In fact, one may verify that each principal ideal $(\gamma_X]$ of level larger than 2 is isomorphic to the finite power set Boolean algebra with $|\gamma_X|$-many elements, as illustrated in Figure 9(b) for the power set Boolean algebra $(2^{TGS}, \subseteq, \cap, \cup, -, \emptyset, TGS)$ generated by the principal ideal $(\gamma_8]$. It is also observable easily, that the principal ideal $(\gamma_1]$ of level 2 is not isomorphic to a Boolean algebra. Indeed, (R, Σ) is not in 1-BCNF.

7.6 Faithful Translations between Schemata and Instances

The previous sections have established two complementary algebraic perspectives of ℓ-Bounded Cardinality Normal Form, one from the perspective of instances and one from that of schemata.

In practical settings, instances i ought to comply with the given set Σ of constraints on the schema. However, typically $\Sigma_i^+ - \Sigma_S^+$ is non-empty and consists of constraints that are satisfied by i but currently not perceived as a business rule. Vice versa, $\Sigma_S^+ - \Sigma_i^+$ is typically also non-empty and consists of currently perceived business rules that are not satisfied by i. As argued in Section 7.1, both cases lead to opportunities for improving data quality, analytics, and database performance.

This motivates the faithful translation of database instances i into schemata S, and vice versa. On one hand, we would mine a cover for the set Σ_i^+ of all CCs and FDs that are satisfied by a given database instance i, a task known as *dependency inference* [49] or *data profiling* [1], with many contributions for FDs such as [54, 67] and little work on CCs [33]. On the other hand, we would compute some representation i_{Σ_S} of a database instance that satisfies those constraints implied by Σ_S and violates those not implied by Σ_S. This problem is known as computing an *Armstrong database* for Σ_S, which has received considerable attention from the research community since the 1980s [5, 25, 37, 48].



(a) Partition lattice with levels of partitions (b) Closure lattice with levels of closed sets

Fig. 10. Lattices associated with the relation and discovered constraints of our running example.

Example 7.39. The relation r from Table 14 is Armstrong for the set Σ of constraints from Example 7.21. As a consequence, the partition lattice L_r of r in Figure 9(a) and the closure lattice L_Σ of Σ in Figure 9(b) are dual to one another modulo the isomorphism defined by $\pi_i \mapsto \gamma_i$ for $i = 0, \ldots, 9$.

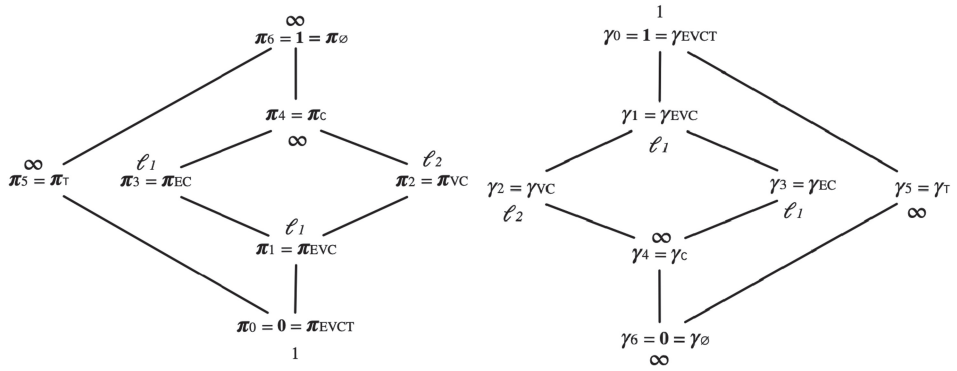
We conclude this section by applying the concepts and results to our running example Hap. First, we illustrate the translation from instance- to schema-based perspective.

Example 7.40. Let r denote the relation from Table 1a. We use the entries from attribute *Time* as tuple identifiers. The elements of the partition lattice L_r associated with r are: $\pi_0 = \mathbf{0} = \pi_T = \pi_{EV} = \pi_{EVC} = \pi_{ET} = \pi_{CT} = \pi_{VT} = \pi_{ECT} = \pi_{EVT} = \pi_{CVT} = \pi_{ECTV} = \{\overline{t_1}; \ldots; \overline{t'_{\ell_2}}\}$, $\pi_1 = \pi_E = \pi_{EC} = \{\overline{t_1, \ldots, t_{\ell_1}}; \overline{t'_1}; \ldots; \overline{t'_{\ell_2}}\}$, $\pi_2 = \pi_V = \pi_{VC} = \{\overline{t_1}; \ldots, \overline{t_{\ell_1}}; \overline{t'_1, \ldots, t'_{\ell_2}}\}$, $\pi_3 = \pi_C = \{\overline{t_1, \ldots, t_{\ell_1}}; \overline{t'_1, \ldots, t'_{\ell_2}}\}$, and $\pi_4 = \mathbf{1} = \pi_\emptyset = \{\overline{t_1, \ldots, t'_{\ell_2}}\}$. The Hasse diagram of the partition lattice is illustrated in Figure 10(a). The levels of the partitions in L_r are $\ell_{\pi_0} = 1$, $\ell_{\pi_1} = \ell_1$, $\ell_{\pi_2} = \ell_2$, $\ell_{\pi_3} = \ell_2$, and $\ell_{\pi_4} = \ell_1 + \ell_2$. It is evident that r is not in ℓ_1-BCNF. For example, the FD $V \to C$ holds on $r[VC]$ but the principal filter $[\pi_2)$ is abnormal. However, r is in ℓ_2-BCNF since the only principal filter with level larger than ℓ_2 is $[\pi_4)$, which is isomorphic to the Boolean algebra with one element.

It is not difficult to see that the set Σ_r of CCs and FDs that hold on the relation r is covered by $E \to C$, $V \to C$, $card(E) \leq \ell_1$, $card(T) \leq \ell_1$, $card(V) \leq \ell_2$, $card(C) \leq \ell_2$, and $card(EV) \leq 1$. Note that we do not consider CCs on the empty set. The closure lattice of Σ_r is illustrated in Figure 10(b), and, in particular, $\gamma_E = \gamma_{EC}$, $\gamma_V = \gamma_{VC}$, and $\gamma_T = \gamma_{EV} = \gamma_{EVCT}$ hold. Dually to the above, the principal ideal $(\gamma_{VC}]$ is above level ℓ_1 and abnormal, and the only principal ideal of level above ℓ_2 is $(\gamma_4]$, which is again isomorphic to the Boolean algebra with one element. Hence, (R, Σ_r) is not in ℓ_1- but it is in ℓ_2-BCNF. □

Finally, we illustrate the translation from schema- to instance-based perspective on our running example.

Example 7.41. Here we start with the set $\Sigma := \Sigma_0$ of CCs and FDs over Hap from Example 3.2. The elements of the closure lattice L_Σ associated with Σ are: $\gamma_0 = \mathbf{1} = \gamma_{ET} = \gamma_{VT} = \gamma_{CT} = \gamma_{ECT} = \gamma_{EVT} = \gamma_{CTV} = \gamma_{EVCT}$, $\gamma_1 = \gamma_{EV} = \gamma_{EVC}$, $\gamma_2 = \gamma_V = \gamma_{VC}$, $\gamma_3 = \gamma_E = \gamma_{EC}$, $\gamma_4 = \gamma_C$, $\gamma_5 = \gamma_T$, $\gamma_6 = \gamma_\emptyset = \mathbf{0}$. The Hasse diagram of the closure lattice is illustrated in Figure 11(b). The levels of closed sets in L_Σ are $\ell_{\gamma_0} = 1$, $\ell_{\gamma_1} = \ell_1$, $\ell_{\gamma_2} = \ell_2$, $\ell_{\gamma_3} = \ell_1$, and $\ell_{\gamma_4} = \ell_{\gamma_5} = \ell_{\gamma_6} = \infty$. It is evident that (Hap, Σ) is not in ℓ_1-BCNF, as for the principal ideal $(\gamma_2]$ of level ℓ_2 the FD $V \to C$ holds on $\Sigma[VC]$ but $(\gamma_2]$ is abnormal. However, (R, Σ) is in ℓ_2-BCNF since the only principal ideals with level larger



(a) Partition lattice with levels of partitions (b) Closure lattice with levels of closed sets

Fig. 11. Lattices associated with Armstrong relation and schema of running example.

than ℓ_2 are $(\gamma_4]$, $(\gamma_5]$, and $(\gamma_6]$, where the first two are isomorphic to the Boolean algebra with two elements, and the last is isomorphic to the Boolean algebra with one element.

Since Σ does not imply any finite upper bounds on the singletons C and T, there is no finite relation over Hap that satisfies exactly those CCs and FDs that are implied by Σ. However, we can still choose a finite representation of an infinite Armstrong relation, such as the following representation r'_Σ:

Event	*Company*	*Venue*	*Time*	*card*
$*$	c_1	$*$	$*$	∞
e_2	c_2	$*$	$*$	2
$*$	$*$	$*$	t_3	∞
$*$	c_4	v_4	$*$	ℓ_2
e_5	c_5	v_5	$*$	ℓ_1

.

Indeed, we can obtain an infinite Armstrong relation r_Σ as follows: Each tuple t' in r'_Σ introduces *card* tuples $t_1, \ldots, t_{card}$ into r_Σ that all have unique values in columns C where $*$ occurs in $t'[C]$, and value v in columns C where value $v \neq *$ occurs in $t'[C]$.

The elements of the partition lattice L_{r_Σ} associated with r_Σ are: $\pi_0 = \mathbf{0} = \pi_{ET} = \pi_{CT} = \pi_{VT} = \pi_{ET} = \pi_{ECT} = \pi_{EVT} = \pi_{CVT} = \pi_{ECTV}$, $\pi_1 = \pi_{ECV}$, $\pi_2 = \pi_V = \pi_{VC}$, $\pi_3 = \pi_E = \pi_{EC}$, $\pi_4 = \pi_C$, $\pi_5 = \pi_T$ and $\pi_6 = \mathbf{1} = \pi_\emptyset$. The Hasse diagram of the partition lattice is illustrated in Figure 11(a). The levels of the partitions in L_{r_Σ} are $\ell_{\pi_0} = 1$, $\ell_{\pi_1} = \ell_1$, $\ell_{\pi_2} = \ell_2$, $\ell_{\pi_3} = \ell_1$, and $\ell_{\pi_4} = \ell_{\pi_5} = \ell_{\pi_6} = \infty$. It is evident that r_Σ is not in ℓ_1-BCNF. For example, the FD $V \rightarrow C$ holds on $r_\Sigma[VC]$ but the principal filter $[\pi_2)$ is abnormal. However, r_Σ is in ℓ_2-BCNF since the only principal filters with level larger than ℓ_2 are $[\pi_4)$, $[\pi_5)$, and $[\pi_6)$ which are all isomorphic to Boolean algebras.

Summary. We have established algebraic characterizations for instances and schemata to be in ℓ-Bounded Cardinality Normal Form, for any given ℓ. From the perspective of instances, every principal filter whose level exceeds that of a given ℓ must be isomorphic to a Boolean algebra in the partition lattice associated with the instance. This subsumes the only previously known special case where $\ell = 1$ and every atomic filter is isomorphic to a Boolean algebra. From the dual perspective of schemata, every principal ideal whose level exceeds that of a given ℓ must be isomorphic to a Boolean algebra in the closure lattice associated with the given set of CCs and FDs.

8 Related Work

Schema design has been studied in data models such as SQL [32], nested relations [50], data warehouses [38], object-orientation [29], semantics [10], temporality [28], the Web [2], uncertainty [44], and graphs [19]. Similar to classical design, our results may be extended to richer data models and higher normal forms such as 4NF [17], Project-Join Normal Form [18], or Inclusion-Dependency Normal Form [42].

Unlike BCNF [4, 6, 69] and 3NF [7, 70] which only use FDs, CCs measure the effort for achieving data consistency. Our algorithms return designs that quantify worst-case update inefficiency and best-case join efficiency. Our concepts also quantify incremental maintenance and join support of materialized views during logical design.

NDs apply classical normalization after horizontal decomposition into blocks where FDs hold [22]. An ND is an expression of the form $X \rightarrow_\ell Y$ expressing that every X-value in an underlying relation r can co-occur with at most ℓ different Y-values in r. That is, every X-value is associated with at most ℓ different Y-values. More formally, for all $t_1, \ldots, t_{\ell+1} \in r$ where $t_1(X) = \ldots = t_{\ell+1}(X)$, there are some $1 \leq i, j \leq \ell + 1$ such that $t_i(Y) = t_j(Y)$. Evidently, an FD $X \rightarrow Y$ is captured as the special case of an ND $X \rightarrow_\ell Y$ where $\ell = 1$, and a CC $card(X) \leq \ell$ is captured as the special case of an ND $X \rightarrow_\ell (R - X)$. Consequently, our condition of an ℓ-Bounded Cardinality Normal Form reads that for every $X \rightarrow_1 Y \in \Sigma^+$ where $Y \not\subseteq X$, we must have $X \rightarrow_\ell (R - X) \in \Sigma^+$. Unfortunately, however, the class of NDs is not finitely axiomatizable [21], so a more general approach via general NDs is not possible, at least not directly by using finite axiomatizations. Indeed, our framework only relies on the combined class of the two special cases of NDs which capture FDs and CCs, respectively. This class is axiomatized by $\mathfrak{Q}$ [24].

Another attempt to quantify data redundancy in logical database design is based on information theory [35]. Here, the authors define the *guaranteed information content* of a schema (R, Σ) for an attribute $A \in R$ as the least amount of information content that may be found in A-columns of instances of R that satisfy Σ. This measure represents the worst case of redundancy in column A over all possible instances. The authors demonstrate that 3NF incurs the least possible redundancy among all dependency-preserving decompositions of a schema. Hence, the guaranteed information content quantifies the sources of data redundancy based on FDs alone, while our work quantifies the level of data redundancy that these sources cause using FDs and CCs. As an illustration for the difference, the guaranteed information content for the attribute C is $1/2$ in both $\mathcal{D}_1$ and $\mathcal{D}_2$ and for all other attributes it is 1 in both decompositions, while the smallest level of data redundancy prevented by $\mathcal{D}_1$ is $1k$ and that prevented by $\mathcal{D}_2$ is $1m$. Hence, based on the information-theoretic characterization of 3NF [35], $\mathcal{D}_1$ and $\mathcal{D}_2$ are indistinguishable. An interesting direction of future work is to develop information-theoretic characterizations of the Bounded Cardinality Normal Forms.

Schema design is not expected to optimize the layout for all instances. Instead, physical tuning kicks in after deploying the logical schema and once reliable data retrieval patterns emerge. These include virtual de-normalization in main memory databases [46], migration to NoSQL [68], and data warehouse designs [38]. Guidelines have been devised to recover 3NF from de-normalised schema [56], and de-normalizing normalised schemata [8]. Information-theoretic justifications exist for fact tables in snowflake schemata [41] and for 3NF [35]. Update inefficiency and join efficiency inform the difficult problem of selecting materialized views [12].

Kojić and Milićev de-normalize by maximizing read benefits while write costs remain under a threshold [34]. Their technique tunes a logical schema (for example, in 3NF) based on specific updates and queries. Hence, their approach is applicable once the database is operational and a mature workload model available. This additional input makes it possible to derive a schema

optimized for the workload [34]. The work in [34] does not use CCs. Our approach is for logical schema normalization where a workload model in terms of frequent update and query operations is not available yet. We use CCs to explore various normal forms with different properties in terms of update inefficiency and join efficiency on the schemata. Hence, the design team can assess different designs based on their properties. We only require a set of attributes, FDs, and CCs as input. As our work brings forward a logical schema, it may provide a different starting point for applying the work in [34] than classical normalization does. For example, once frequent updates and queries have emerged, we may apply [34] to the output of Algorithm 2 rather than the 3NF schema (Hap, Σ_0).

Recent approaches to schema design and evolution for NoSQL databases [59] are driven by a query workload, and NoSQL schemata are thus de-normalized. We are unaware of work for logical NoSQL schema design for CCs and FDs.

"Cardinality constraints are one of the most important kinds of constraint in conceptual modeling" [51]. "CCs correspond to very common semantic rules on relationships and their formal definition at the conceptual level improves significantly the completeness of data description" [40]. Surprisingly, our way of applying CCs to logical schema design has not been observed before. CCs were introduced in Chen's seminal ER paper [10], and have been studied in data models such as semantic [43], Web [20], spatial and temporal [14], and uncertain models [58]. They are part of major languages for data and knowledge modeling, including UML, EER, ORM, XSD (such as `maxOccurs`), or OWL (such as `owl:maxCardinality`) [23]. They have also been used in data cleaning [11], database testing [9], query answering [13], and reverse engineering [63].

Violations of dependencies by dirty data motivate relaxed notions such as approximate keys and FDs [26, 36]. When mining from (dirty) data, approximate notions may improve the recall but worsen the precision of identifying meaningful rules. The CC $card(X) \leq \ell$ may be seen as an approximate key permitting up to ℓ duplicates. For small ℓ, we may therefore view our work as extending classical schema design to approximate keys, offering some robustness for dirty data.

The algebraic characterization of relations in ℓ-Bounded Cardinality Normal Form extend that of relations in BCNF by Lee [39] for the special case where $\ell = 1$. We further established a dual algebraic characterization of schemata in ℓ-Bounded Cardinality Normal Form using principal ideals of closed attribute sets. Note that the research literature has brought forward many other treatments of FDs and their associated normal forms from an algebraic perspective [15, 16, 27]. However, none of these has addressed CCs or the new Bounded Cardinality Normal Form.

9 Conclusion and Future Work

We introduced the first logical schema design framework that measures update inefficiency and join efficiency, based on integrity constraints alone. This is possible by the new notion of level-ℓ data redundancy, which is determined by the upper bounds of CCs at schema design time. We showed that for every schema there is a smallest integer ℓ for which it is in ℓ-Bounded Cardinality Normal Form, exhibiting instances (i) that prevent level ℓ data redundancy, (ii) for which level ℓ update inefficiency is required to maintain data consistency, and (iii) permit level ℓ join efficiency. We developed algorithms for schema design, and illustrated experimentally how they reduce the levels of update inefficiency with tradeoffs in the size of output designs. We also showed experimentally how these levels quantify the suitability of schema designs and materialized views for the performance of specific updates and joins on instances of the designs. Our framework uses domain knowledge about CCs to advance logical schema design. In an effort to support practitioners in their never-ending task of optimizing data quality, the insight generated from analytics, and database performance, we used lattice theory to established new foundations for aligning database instances with the business rules that govern them. In particular, we showed that an instance

(schema, respectively) is in ℓ-Bounded Cardinality Normal Form if and only if every principal filter (ideal, respectively) of level larger than ℓ is isomorphic to a Boolean algebra. Hence, we can use deviations between the instances and schemata to improve the quality of database instances and the performance of database operations.

Future work will address more expressive constraints and data models. For example, it would be interesting to study the interaction between CCs and tuple-generating dependencies such as multivalued or join dependencies, for which no axiomatization has been proposed yet. Likewise, generalizing our ideas to data models where schemata are optional would be interesting, such as graph models [60, 62]. Recently, parameterizations of BCNF and 3NF have been proposed, using the number of minimal keys [69] and FDs [70], respectively. Combining that proposals with CCs and the ℓ-Bounded Cardinality Normal Form would be interesting in breaking further ties between schemata, and minimizing operational overheads on instances of the schema designs. Finally, as seen by our experiment with automated data warehouse designs, self-tuning database design [55] is worth perusing as a research direction.

Acknowledgments

The authors would also like to thank the anonymous referees for their valuable comments and helpful suggestions.

References

[1] Ziawasch Abedjan, Lukasz Golab, Felix Naumann, and Thorsten Papenbrock. 2018. *Data Profiling*. Morgan & Claypool Publishers.
[2] Marcelo Arenas. 2006. Normalization theory for XML. *SIGMOD Rec.* 35, 4 (2006), 57–64.
[3] Catriel Beeri and Philip A. Bernstein. 1979. Computational problems related to the design of normal form relational schemas. *ACM Trans. Database Syst.* 4, 1 (1979), 30–59.
[4] Catriel Beeri, Philip A. Bernstein, and Nathan Goodman. 1978. A sophisticate's introduction to database normalization theory. In *Proceedings of the 4th International Conference on Very Large Data Bases*. 113–124.
[5] Catriel Beeri, Martin Dowd, Ronald Fagin, and Richard Statman. 1984. On the structure of armstrong relations for functional dependencies. *J. ACM* 31, 1 (1984), 30–46.
[6] Philip A. Bernstein and Nathan Goodman. 1980. What does Boyce-Codd normal form do?. In *Proceedings of the 6th International Conference on Very Large Data Bases*. 245–259.
[7] Joachim Biskup, Umeshwar Dayal, and Philip A. Bernstein. 1979. Synthesizing independent database schemas. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. 143–151.
[8] Douglas B. Bock and John F. Schrage. 2002. Denormalization guidelines for base and transaction tables. *ACM SIGCSE Bull.* 34, 4 (2002), 129–133.
[9] Nicolas Bruno, Surajit Chaudhuri, and Dilys Thomas. 2006. Generating queries with cardinality constraints for DBMS testing. *IEEE Trans. Knowl. Data Eng.* 18, 12 (2006), 1721–1725.
[10] Peter Chen. 1976. The entity-relationship model-toward a unified view of data. *ACM Trans. Database Syst.* 1, 1 (1976), 9–36.
[11] Wenguang Chen, Wenfei Fan, and Shuai Ma. 2009. Incorporating cardinality constraints and synonym rules into conditional functional dependencies. *Inf. Process. Lett.* 109, 14 (2009), 783–789.
[12] Rada Chirkova and Jun Yang. 2012. Materialized views. *Found. Trends Databases* 4, 4 (2012), 295–405.
[13] Graham Cormode, Divesh Srivastava, Entong Shen, and Ting Yu. 2012. Aggregate query answering on possibilistic data with cardinality constraints. In *Proceedings of the IEEE 28th International Conference on Data Engineering (ICDE 2012)*. 258–269.
[14] Faiz Currim and Sudha Ram. 2008. Conceptually modeling windows and bounds for space and time in database constraints. *Commun. ACM* 51, 11 (2008), 125–129.
[15] János Demetrovics, G. Hencsey, Leonid Libkin, and Ilya B. Muchnik. 1992. Normal form relation schemes: A new characterization. *Acta Cybern.* 10, 3 (1992), 141–153.
[16] János Demetrovics, Leonid Libkin, and Ilya B. Muchnik. 1992. Functional dependencies in relational databases: A lattice point of view. *Discret. Appl. Math.* 40, 2 (1992), 155–185.
[17] Ronald Fagin. 1977. Multivalued dependencies and a new normal form for relational databases. *ACM Trans. Database Syst.* 2, 3 (1977), 262–278.

[18] Ronald Fagin. 1979. Normal forms and relational database operators. In *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. 153–160.
[19] Wenfei Fan, Yinghui Wu, and Jingbo Xu. 2016. Functional dependencies for graphs. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016*. 1843–1857.
[20] Flavio Ferrarotti, Sven Hartmann, and Sebastian Link. 2013. Efficiency frontiers of XML cardinality constraints. *Data Knowl. Eng.* 87 (2013), 297–319.
[21] John Grant and Jack Minker. 1985. Inferences for numerical dependencies. *Theor. Comput. Sci.* 41 (1985), 271–287.
[22] John Grant and Jack Minker. 1985. Normalization and axiomatization for numerical dependencies. *Inf. Control.* 65, 1 (1985), 1–17.
[23] Neil Hall, Henning Köhler, Sebastian Link, Henri Prade, and Xiaofang Zhou. 2015. Cardinality constraints on qualitatively uncertain data. *Data Knowl. Eng.* 99 (2015), 126–150.
[24] Sven Hartmann. 2003. Reasoning about participation constraints and Chen's constraints. In *Database Technologies 2003, Proceedings of the 14th Australasian Database Conference, ADC 2003*. 105–113.
[25] Sven Hartmann, Markus Kirchberg, and Sebastian Link. 2012. Design by example for SQL table definitions with functional dependencies. *VLDB J.* 21, 1 (2012), 121–144.
[26] Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, and Hannu Toivonen. 1999. TANE: An efficient algorithm for discovering functional and approximate dependencies. *Comput. J.* 42, 2 (1999), 100–111.
[27] Jouni Järvinen. 2007. Pawlak's information systems in terms of Galois connections and functional dependencies. *Fundam. Informaticae* 75, 1–4 (2007), 315–330.
[28] Christian S. Jensen, Richard T. Snodgrass, and Michael D. Soo. 1996. Extending existing dependency theory to temporal databases. *IEEE Trans. Knowl. Data Eng.* 8, 4 (1996), 563–582.
[29] Vitaliy L. Khizder and Grant E. Weddell. 2003. Reasoning about uniqueness constraints in object relational databases. *IEEE Trans. Knowl. Data Eng.* 15, 5 (2003), 1295–1306.
[30] Henning Koehler and Sebastian Link. 2024. Entity/relationship profiling. In *Proceedings of the 40th IEEE International Conference on Data Engineering, ICDE 2024*. 5393–5396.
[31] Henning Köhler. 2006. Finding faithful Boyce-Codd normal form decompositions. In *Proceedings of the 2nd International Conference on Algorithmic Aspects in Information and Management, AAIM 2006*. 102–113.
[32] Henning Köhler and Sebastian Link. 2016. SQL schema design: Foundations, normal forms, and normalization. In *Proceedings of the 2016 International Conference on Management of Data*. 267–279.
[33] Henning Köhler, Sebastian Link, and Xiaofang Zhou. 2016. Discovering meaningful certain keys from incomplete and inconsistent relations. *IEEE Data Eng. Bull.* 39, 2 (2016), 21–37.
[34] Nemanja Kojic and Dragan Milicev. 2020. Equilibrium of redundancy in relational model for optimized data retrieval. *IEEE Trans. Knowl. Data Eng.* 32, 9 (2020), 1707–1721.
[35] Solmaz Kolahi and Leonid Libkin. 2010. An information-theoretic analysis of worst-case redundancy in database design. *ACM Trans. Database Syst.* 35, 1 (2010), 5:1–5:32.
[36] Sebastian Kruse and Felix Naumann. 2018. Efficient discovery of approximate dependencies. *Proc. VLDB Endow.* 11, 7 (2018), 759–772.
[37] Warren-Dean Langeveldt and Sebastian Link. 2010. Empirical evidence for the usefulness of armstrong relations in the acquisition of meaningful functional dependencies. *Inf. Syst.* 35, 3 (2010), 352–374.
[38] Jens Lechtenbörger and Gottfried Vossen. 2003. Multidimensional normal forms for warehouse design. *Inf. Syst.* 28, 5 (2003), 415–434.
[39] Tony T. Lee. 1983. An algebraic theory of relational databases. *Bell Syst. Tech. J.* 62, 10 (1983), 3159–3204.
[40] Maurizio Lenzerini and Gaetano Santucci. 1983. Cardinality constraints in the entity-relationship model. In *Proceedings of the 3rd International Conference on Entity-Relationship Approach (ER'83)*. 529–549.
[41] Mark Levene and George Loizou. 2003. Why is the snowflake schema a good data warehouse design? *Inf. Syst.* 28, 3 (2003), 225–240.
[42] Mark Levene and Millist W. Vincent. 2000. Justification for inclusion dependency normal form. *IEEE Trans. Knowl. Data Eng.* 12, 2 (2000), 281–291.
[43] Stephen W. Liddle, David W. Embley, and Scott N. Woodfield. 1993. Cardinality constraints in semantic data models. *Data Knowl. Eng.* 11, 3 (1993), 235–270.
[44] Sebastian Link and Henri Prade. 2019. Relational database schema design for uncertain data. *Inf. Syst.* 84 (2019), 88–110.
[45] Sebastian Link and Ziheng Wei. 2021. Logical schema design that quantifies update inefficiency and join efficiency. In *SIGMOD '21: Proceedings of the International Conference on Management of Data*. 1169–1181.
[46] Zezhou Liu and Stratos Idreos. 2016. Main memory adaptive denormalization. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016*. 2253–2254.
[47] David Maier. 1983. *The Theory of Relational Databases*. Computer Science Press.

[48] Heikki Mannila and Kari-Jouko Räihä. 1986. Design by example: An application of armstrong relations. *J. Comput. Syst. Sci.* 33, 2 (1986), 126–141.
[49] Heikki Mannila and Kari-Jouko Räihä. 1987. Dependency inference. In *VLDB'87, Proceedings of the 13th International Conference on Very Large Data Bases*. 155–158.
[50] Wai Yin Mok, Yiu-Kai Ng, and David W. Embley. 1996. A normal form for precisely characterizing redundancy in nested relations. *ACM Trans. Database Syst.* 21, 1 (1996), 77–106.
[51] Antoni Olivé. 2007. Cardinality constraints. In *Conceptual Modeling of Information Systems*. Springer, 83–102.
[52] Sylvia L. Osborn. 1979. Testing for existence of a covering Boyce-Codd normal form. *Inf. Process. Lett.* 8, 1 (1979), 11–14.
[53] Thorsten Papenbrock, Jens Ehrlich, Jannik Marten, Tommy Neubert, Jan-Peer Rudolph, Martin Schönberg, Jakob Zwiener, and Felix Naumann. 2015. Functional dependency discovery: An experimental evaluation of seven algorithms. *PVLDB* 8, 10 (2015), 1082–1093.
[54] Thorsten Papenbrock and Felix Naumann. 2016. A hybrid approach to functional dependency discovery. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016*. 821–833.
[55] Thorsten Papenbrock and Felix Naumann. 2017. Data-driven schema normalization. In *Proceedings of the 20th International Conference on Extending Database Technology, EDBT 2017*. 342–353.
[56] Jean-Marc Petit, Farouk Toumani, Jean-François Boulicaut, and Jacques Kouloumdjian. 1996. Towards the reverse engineering of denormalized relational databases. In *Proceedings of the 12th International Conference on Data Engineering*. 218–227.
[57] Jorma Rissanen. 1977. Independent components of relations. *ACM Trans. Database Syst.* 2, 4 (1977), 317–325.
[58] Tania Roblot, Miika Hannula, and Sebastian Link. 2018. Probabilistic cardinality constraints. *VLDB J.* 27, 6 (2018), 771–795.
[59] Stefanie Scherzinger and Sebastian Sidortschuck. 2020. An empirical study on the design and evolution of NoSQL database schemas. In *Proceedings of the 39th International Conference on Conceptual Modeling, ER 2020*. 441–455.
[60] Philipp Skavantzos and Sebastian Link. 2023. Normalizing property graphs. *Proc. VLDB Endow.* 16, 11 (2023), 3031–3043.
[61] Philipp Skavantzos and Sebastian Link. 2025. Entity/relationship graphs: Principled design, modeling, and data integrity management of graph databases. *Proc. ACM Manag. Data* 3, 1 (2025), 40:1–40:26.
[62] Philipp Skavantzos and Sebastian Link. 2025. Third and Boyce-Codd normal form for property graphs. *VLDB J.* 34, 2 (2025), 23.
[63] Christian Soutou. 1998. Relational database reverse engineering: Algorithms to extract cardinality constraints. *Data Knowl. Eng.* 28, 2 (1998), 161–207.
[64] Bernhard Thalheim. 1992. Fundamentals of cardinality constraints. In *Entity-Relationship Approach - ER'92, Proceedings of the 11th International Conference on the Entity-Relationship Approach*. 7–23.
[65] Bernhard Thalheim. 2000. *Entity-Relationship Modeling - Foundations of Database Technology*. Springer.
[66] Millist W. Vincent. 1999. Semantic foundations of 4NF in relational database design. *Acta Inf.* 36, 3 (1999), 173–213.
[67] Ziheng Wei and Sebastian Link. 2023. Towards the efficient discovery of meaningful functional dependencies. *Inf. Syst.* 116 (2023), 102224.
[68] Jaejun Yoo, Ki-Hoon Lee, and Young-Ho Jeon. 2018. Migration from RDBMS to NoSQL using column-level denormalization and atomic aggregates. *J. Inf. Sci. Eng.* 34, 1 (2018), 243–259.
[69] Zhuoxing Zhang, Wu Chen, and Sebastian Link. 2023. Composite object normal forms: Parameterizing Boyce-Codd normal form by the number of minimal keys. *Proc. ACM Manag. Data* 1, 1 (2023), 13:1–13:25.
[70] Zhuoxing Zhang and Sebastian Link. 2025. Synthesizing third normal form schemata that minimize integrity maintenance and update overheads: Parameterizing 3NF by the numbers of minimal keys and functional dependencies. *Proc. ACM Manag. Data* 3, 3 (2025), 225:1–225:25.

Received 29 April 2022; revised 19 March 2025; accepted 7 June 2025